Universidade da Maia

Licenciatura: Informática - Unidade Curricular: Sistemas Inteligentes

Análise Preditiva de Gorjetas de Táxis

Relatório do Projeto de Sistemas Inteligentes

2025/2026

Grupo:

Felipe Castilho, nº47152

Juliana Moreira, nº47188

Marta Vieira, nº46756

Índice

1. Introdução

1.1. Organização do Trabalho

1.2. Metodologia e Pipeline de Análise Preditiva

2. Análise Exploratória de Dados e Pré-processamento

2.1. O Dataset e as Suas Variáveis

2.2. Engenharia de Atributos e Pré-processamento

2.2.1 Tratamento de Dados

2.2.2 Definição dos *Targets* de Modelação

2.2.3 Análise Exploratória de Dados (Pós-Processamento e Divisão 60/20/20)

3. Modelos Clássicos de Machine Learning

3.1. Treino, Validação e Teste dos Modelos

3.1.1 Treino e Otimização de Modelos

3.1.2 Threshold de Classificação

3.2. Resultados do Modelo Random Forest

3.2.1 tip_given

3.2.2 tig_high

3.2.3. tip_amount

3.2.4. Validação Externa no Weka (Random Forest)

3.3. Resultados do Modelo Logistic Regression

3.3.1. tip_given

3.3.2. tip_high

3.3.3. Validação Externa no Weka (Logistic Regression)

3.4. Resultados dos Restantes Modelos Supervisionados

3.5. Validação Externa no Weka – Modelos Árvore de Decisão, Naive Bayes, SVM e MLP

4. Modelos Neuronais e Aprendizagem Não Supervisionada

4.1. Arquitetura e Treino de Redes Neuronais

4.1.1. Arquitetura do Perceptron Multicamadas (MLP)

4.1.2. Regularização

4.2. Deep MLP

4.2.1. Arquitetura e Hiperparâmetros

4.3. KMeans

4.4. DBSCAN

4.5. Autoencoder

5. Comparação de Modelos e Conclusões

5.1. Machine Learning vs Deep Learning

5.1.2. Modelos Supervisionados vs Não Supervisionados

5.2. Análise Comparativa de Desempenho

5.3. Conclusões

6. Referências

1. Introdução

1.1. Organização do Trabalho

O trabalho está estruturado em quatro fases distintas, abrangendo desde a análise exploratória e tratamento de dados até a implementação e comparação de modelos preditivos. Cada fase foi concebida para abordar um aspeto crucial do processo de construção de um sistema de inteligência artificial, garantindo uma abordagem sistemática e robusta. A execução do projeto foi realizada em ambiente Python, com o auxílio do Docker para gestão de dependências e execução. A estrutura de diretórios do projeto reflete esta organização, com um diretório src com os scripts para cada fase, um diretório *outputs* para armazenar os resultados intermédios e finais (incluindo relatórios de dados, divisões de treino/validação/teste e modelos), e um diretório models para outras figuras e resultados específicos. Adicionalmente, foram feitos ficheiros ZIP com *outputs* de diferentes tamanhos de amostra, permitindo uma análise da escalabilidade e consistência dos resultados. Um ficheiro *README* detalha as instruções de implementação, organização e execução do projeto.

1.2. Metodologia e *Pipeline* de Análise Preditiva

Este projeto visa desenvolver um *pipeline* completo de inteligência artificial focado na previsão do comportamento de gorjetas em táxis na cidade de Nova Iorque. A metodologia adotada baseia-se numa divisão rigorosa do dataset em subconjuntos de **60% para treino, 20% para validação e 20% para teste**. Esta proporção é uma prática comum em *machine learning* para garantir que os modelos são treinados com dados suficientes, otimizados em um conjunto independente e avaliados em dados nunca antes vistos, prevenindo o *overfitting* e fornecendo uma estimativa realista do desempenho em novos dados. Para assegurar a significância estatística e avaliar a escalabilidade dos modelos, foram utilizadas amostras de **1.000.000 de registos** como base para este relatório. É importante notar que o projeto também incluiu análises com amostras de 500.000 e 2.000.000 de registos, cujos resultados, embora não detalhados neste relatório, corroboraram a consistência dos *outputs* finais. Sendo que estão presentes em ZIP na pasta principal do projeto.

Os objetivos principais deste trabalho são: prever a ocorrência de uma gorjeta (“Vai haver gorjeta?”) e estimar o valor dessa gorjeta (“Qual o valor da gorjeta?”). A abordagem para estas duas questões envolve diferentes tipos de modelagem, nomeadamente classificação e regressão, respetivamente. Embora este relatório se foque nos principais resultados e gráficos, o projeto completo contém uma grande quantidade de informações adicionais, incluindo gráficos e ficheiros CSV, disponíveis na pasta do projeto para uma análise mais aprofundada.

2. Análise Exploratória de Dados e Pré-processamento

2.1. O Dataset e as Suas Variáveis

O *dataset* utilizado para esta análise provém dos registos de viagens de táxi da *NYC Taxi and Limousine Commission* (TLC) [1], uma fonte de dados utilizada em estudos de transporte e *machine learning*. Este *dataset*, contém informações detalhadas sobre cada viagem, incluindo datas e horas de recolha e entrega, localizações, distâncias percorridas, valores de tarifa, gorjetas,

e tipos de pagamento. A compreensão aprofundada destas variáveis é fundamental para a construção de modelos preditivos eficazes.

A análise inicial das variáveis revelou padrões importantes no comportamento das viagens e gorjetas. O **valor da viagem** (fare_amount) concentra-se predominantemente entre **5 e 10 dólares**, indicando que a maioria das viagens é de curta a média distância. A **gorjeta** (tip_amount), por sua vez, apresenta uma distribuição mais variada, com **picos notáveis em 1 e 2 dólares**. Esta observação sugere que, embora a maioria das gorjetas seja de valores baixos, a sua variabilidade pode ser influenciada por diversos fatores.

Em termos de distância, a maioria das viagens é **curta e barata**, com **0.5 a 2 milhas** de percurso. A **duração** da viagem segue um padrão semelhante, com um pico entre **5 e 15 minutos**. A maioria das viagens transporta **1 passageiro**, e a **velocidade média** situa-se entre **7 e 12 milhas por hora**. Curiosamente, o **tipo de tarifa** (rate_code_id) não apresenta variações significativas, sendo predominantemente **Standard**.

Observou-se também que os **dias da semana** registam um volume de viagens superior aos **fins de semana**. Um aspeto crucial é o facto de uma grande proporção de clientes ter dado gorjeta, o que resulta num ***dataset* desbalanceado** para o objetivo de prever a ocorrência de gorjetas. O valor médio da gorjeta é de 2 dólares, apesar da moda ser 1 dólar, indicando que algumas gorjetas mais elevadas exercem uma **grande influência na mediana e na média**. Da mesma forma, o valor médio da tarifa é de 9.8 dólares, com uma moda de 6.5 dólares, exibindo um padrão similar ao das gorjetas.

As figuras a seguir ilustram estas características do *dataset*:

Descrição das Variáveis do Dataset

Variável	Tipo	Descrição
fare_amount	Quantitativa Contínua	Valor base da corrida ($)
tip_amount	Quantitativa Contínua	Valor da gorjeta — TARGET regressão ($)
trip_distance	Quantitativa Contínua	Distância da viagem (milhas)
trip_duration_min	Quantitativa Contínua	Duração da viagem (min) — derivada
avg_speed	Quantitativa Contínua	Velocidade média (mph) — derivada
passenger_count	Quantitativa Discreta	Número de passageiros (1–6)
pickup_hour	Quantitativa Discreta	Hora de início da viagem (0–23) — derivada
pickup_weekday	Quantitativa Discreta	Dia da semana (0–6) — derivada
payment_type	Qualitativa Nominal	1=Credit card, 2=Cash, 3=No charge, 4=Dispute, 5=Unknown, 6=Voided trip
RatecodeID	Qualitativa Ordinal	1=Standard, 2=JFK, 3=Newark, 4=Nassau/Westchester, 5=Negotiated fare, 6=Group ride
pickup_weekend	Qualitativa Binária	Fim de semana? (0/1) — derivada
tip_given	Qualitativa Binária	Deu gorjeta? (0/1) — TARGET classificação (baseline)
tip_high	Qualitativa Binária	Gorjeta alta? (0/1) — ≥ P75 de tip_amount

Figura 2.1: Tabela de Descrição das Variáveis do Dataset Utilizadas. Esta tabela detalha as variáveis selecionadas do *dataset*, fornecendo uma visão geral de sua natureza e relevância para a análise preditiva.

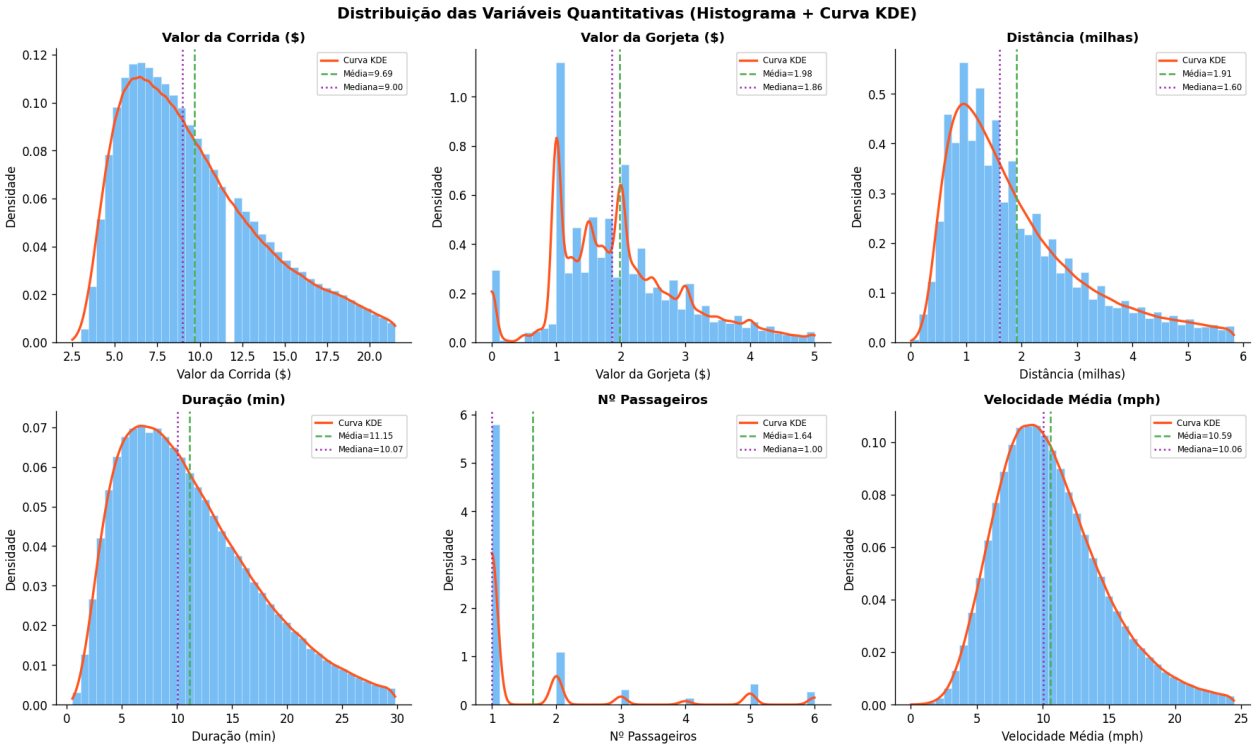


Figura 2.2/3: Distribuição de Variáveis Quantitativas no Dataset. Este(s) gráfico(s) apresenta(m) a distribuição de frequência do valor da viagem, valor da gorjeta e distância percorrida em milhas, destacando os intervalos mais comuns para cada uma. Além disso, demonstra a distribuição da duração da viagem em minutos, o número de passageiros e a velocidade média em milhas por hora, revelando padrões de comportamento dos utilizadores.

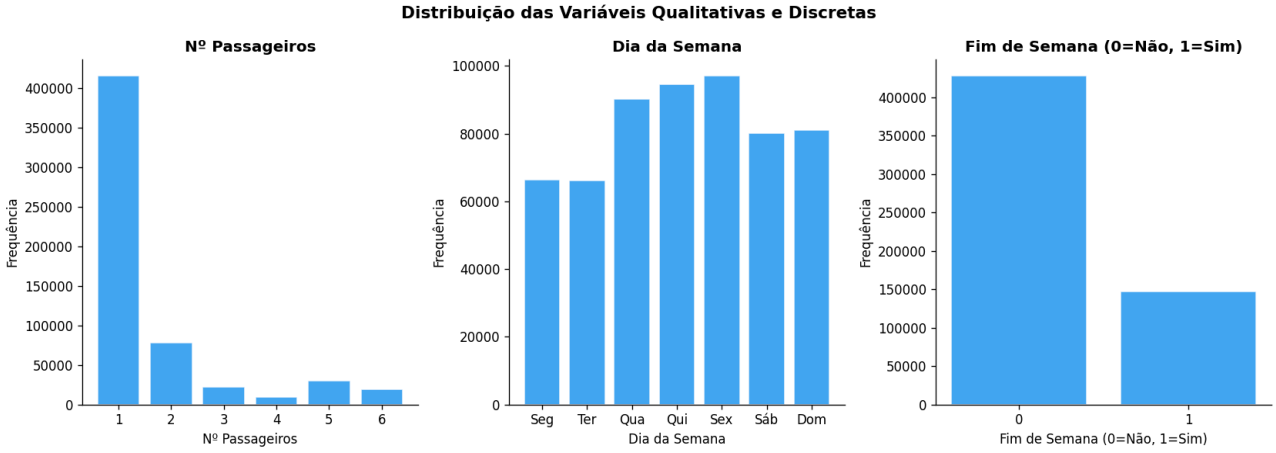


Figura 2.4: Distribuição de Variáveis Qualitativas no Dataset. Este gráfico mostra a frequência de ocorrências para variáveis categóricas como o número de passageiros, dias da semana e fins de semana, fornecendo insights sobre os padrões de uso do serviço de táxi.

2.2. Engenharia de Atributos e Pré-processamento

O pré-processamento de dados é uma etapa crítica em qualquer projeto de *machine learning*, visando limpar, transformar e preparar os dados para a modelagem. Neste projeto, foram aplicadas diversas técnicas de tratamento de dados e engenharia de atributos para otimizar o desempenho dos modelos preditivos.

2.2.1. Tratamento de Dados

Uma observação fundamental no *dataset* da TLC é que os pagamentos em dinheiro **não registam gorjetas eletrónicas**. A inclusão destas viagens, onde tip_amount seria zero por padrão, introduziria um **ruído sistemático** nos *targets* de previsão, distorcendo a capacidade do modelo de aprender padrões reais de gorjetas. Consequentemente, o dataset foi **filtrado** para incluir apenas viagens com **pagamento por cartão**. Esta decisão, embora resulte num desequilíbrio de classes para o *target* tip_given (aproximadamente 96.5% de viagens com gorjeta), reflete a realidade dos pagamentos eletrónicos e permite focar a modelagem em cenários onde a gorjeta é uma decisão explícita do cliente.

As etapas de tratamento de dados incluíram:

- **Remoção de linhas incompletas e mal formadas:** Garantir a integridade dos dados através da eliminação de registos corrompidos ou com valores ausentes críticos.
- **Aplicação de IQR (Intervalo Interquartil):** Utilização do método IQR para identificar e tratar *outliers* nas variáveis principais como tarifa, gorjeta, distância e duração da viagem. Este método é eficaz para mitigar o impacto de valores extremos que podem distorcer a análise estatística e o treino do modelo.
- **Aplicação de regras de domínio:** Implementação de regras baseadas no conhecimento do negócio para filtrar dados inconsistentes ou irrealistas. Estas regras incluíram:
  - passenger_count entre 1 e 6.
  - trip_distance entre 0 e 100 milhas.
  - fare_amount entre 0 e 300 dólares.
  - trip_duration entre 30 segundos e 180 minutos.
  - tip_amount maior que 0 (após a filtragem por tipo de pagamento).
  - pick_up_time e drop_off_time não nulos.
  - payment_type não nulo.

2.2.2. Definição dos Targets de Modelação

O projeto definiu três *targets* distintos para a modelagem preditiva, cada um com os seus desafios e objetivos específicos:

- **tip_given (Baseline):** Este é um problema de classificação binária que visa prever se uma gorjeta foi dada ou não. As classes são altamente desequilibradas (aproximadamente 96% com gorjeta / 4% sem gorjeta), o que representa um desafio significativo para os modelos, que tendem a favorecer a classe maioritária.
- **tip_high (Target Principal):** Este *target* de classificação binária procura distinguir entre gorjetas altas e baixas. Foi definido como gorjeta acima do percentil 75 nas viagens com gorjeta registada (aproximadamente tip_amount $\geq$ 2.45 dólares). A criação deste *target*

visa abordar o desequilíbrio do tip_given e focar na previsão de gorjetas de maior valor. As classes são mais equilibradas (aproximadamente 25% gorjeta alta / 75% gorjeta baixa), tornando a modelagem mais viável.

- **tip_amount (Regressão):** Este *target* de regressão tem como objetivo prever o valor exato da gorjeta. É um problema mais complexo, pois exige que o modelo estime um valor contínuo em vez de uma classe binária.

2.2.3. Análise Exploratória de Dados (Pós-processamento e Divisão 60/20/20)

Após o pré-processamento e a definição dos *targets*, uma análise exploratória mais aprofundada foi realizada nos dados já divididos em conjuntos de treino, validação e teste. Esta etapa é crucial para entender as características dos dados que serão alimentados aos modelos.

Estatísticas Descritivas — Média, Mediana, Moda, Mín, Máx

Variável	Tipo	Média	Mediana	Moda	Mín	Máx	Desvio P.
fare_amount	Quant.	9.82	9.0	6.5	2.5	26.5	4.332
tip_amount	Quant.	2.007	1.86	1.0	0.0	5.9	1.037
trip_distance	Quant.	1.948	1.6	0.9	0.01	6.27	1.271
trip_duration_min	Quant.	11.351	10.15	7.683	0.517	33.983	6.364
passenger_count	Quant.	1.64	1.0	1.0	1.0	6.0	1.306
avg_speed	Quant.	10.765	10.105	9.0	0.033	452.308	4.508
pickup_hour	Quant.	13.815	15.0	19.0	0.0	23.0	6.379
tip_given	Qualit.	0.965	1.0	1.0	0.0	1.0	0.183
tip_high	Qualit.	0.252	0.0	0.0	0.0	1.0	0.434

Figura 2.5: Tabela de Estatísticas Sumárias para o Dataset. Esta tabela apresenta as principais métricas descritivas (média, desvio padrão, mínimo, máximo, quartis) das *features* numéricas do dataset, fornecendo uma visão quantitativa da distribuição dos dados.

O *pie chart* abaixo indica que **96.5% das viagens pagas por cartão têm gorjeta**, reforçando o desequilíbrio de classes para o *target* tip_given. O Histograma da distribuição de gorjetas sugere que esta é **semelhante ao longo dos dias da semana**, o que pode simplificar a modelagem ao indicar que o dia da semana pode não ser um fator de grande variância para a ocorrência de gorjetas.

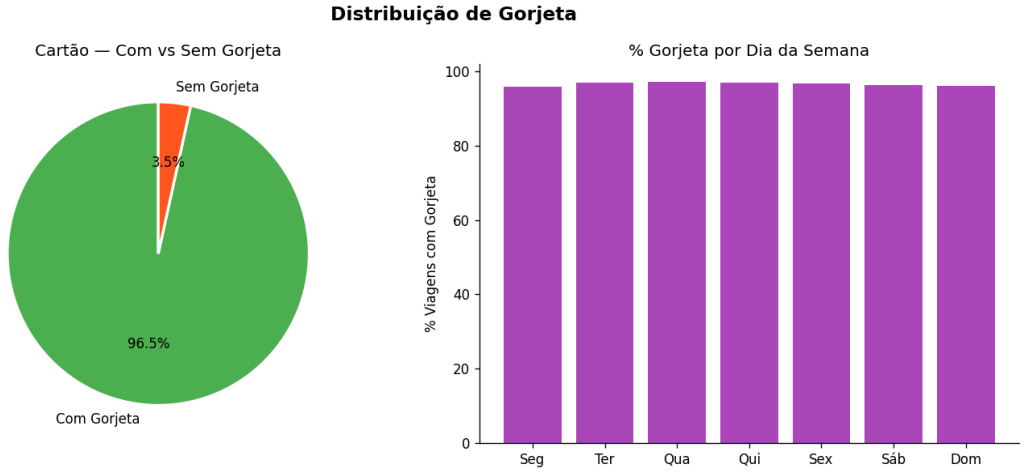


Figura 2.6: Distribuição de tip_amount. O *pie chart* indica a quantidade (em %) de viagens pagas com gorjeta e, por sua vez, sem. O histograma mostra a frequência dos valores de gorjeta.

Devido ao forte desequilíbrio de classes, a **probabilidade de gorjeta por hora é sempre elevada**, o que significa que a maioria das viagens, independentemente da hora, tende a receber gorjeta. O valor médio da gorjeta mantém-se **estável durante todo o dia**, com uma pequena diminuição entre as **5 e 7 da manhã**, possivelmente refletindo um período de menor movimento ou características diferentes dos passageiros.

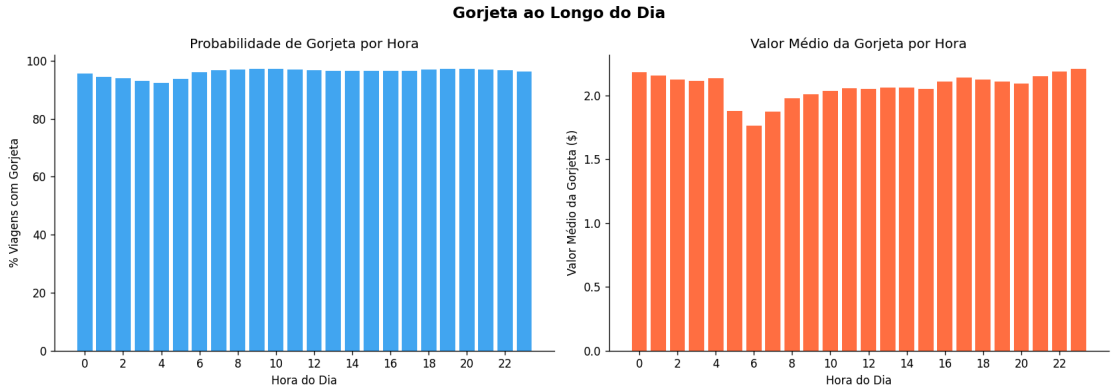


Figura 2.7: Gorjetas ao Longo do Dia. Este(s) gráfico(s) de linha ilustra(m) a variação de viagens e a sua probabilidade de gorjeta (em %) e do valor médio da gorjeta, respetivamente. Ambas relativamente ao longo das horas do dia, destacando a sua estabilidade e a ligeira queda matinal.

Foi observada uma **correlação positiva clara entre o preço da viagem (tarifa) e o valor da gorjeta**. Isto implica que, quanto maior a tarifa, maior a gorjeta esperada, uma relação intuitiva que os modelos devem ser capazes de capturar. A **distância e a duração da viagem** também se correlacionam **positivamente** com a gorjeta, mas com uma **maior dispersão**, sugerindo que outros fatores, como o contexto da viagem ou a hora do dia, podem ter uma influência mais pronunciada nestas relações.

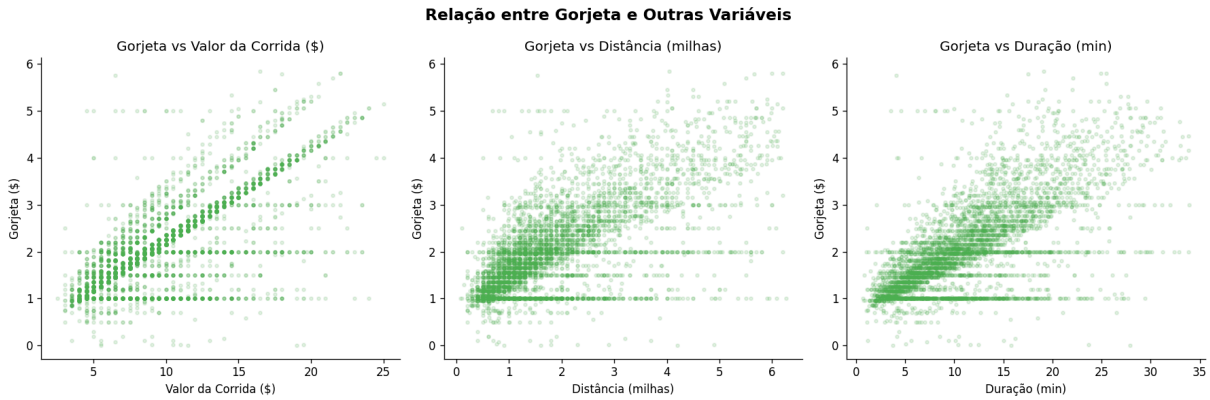


Figura 2.8: Relação entre Gorjeta e Outras Variáveis. Este conjunto de gráficos de dispersão explora as relações entre o valor da gorjeta e variáveis como valor da viagem, distância e duração da viagem, visualizando as correlações e dispersões.

A matriz de correlação confirmou que **o valor da gorjeta, a distância da viagem e o tempo da viagem** são as *features* mais correlacionadas com **o valor da tarifa (viagem)**. Esta informação é vital para a seleção de *features* e para a compreensão das interdependências entre as variáveis do *dataset*.

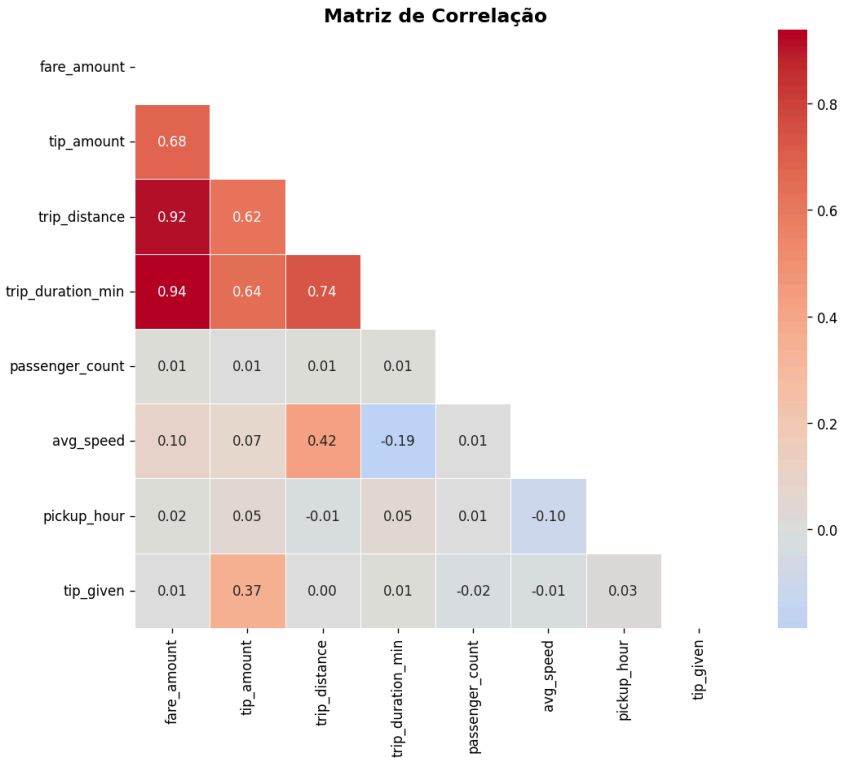


Figura 2.9: Matriz de Correlação entre Features Numéricas. Esta matriz de calor visualiza os coeficientes de correlação entre todas as *features* numéricas, destacando as relações lineares mais fortes e mais fracas.

3. Modelos Clássicos de Machine Learning

Nesta fase, foram explorados e avaliados diversos modelos clássicos de *machine learning* para os *targets* de classificação e regressão definidos.

3.1. Treino, Validação e Teste dos Modelos

A divisão do dataset em conjuntos de treino, validação e teste é uma prática padrão e essencial no desenvolvimento de modelos de *machine learning* para garantir a generalização e evitar o *overfitting*. A proporção de **60% para treino, 20% para validação e 20% para teste** foi cuidadosamente escolhida:

- **60% de treino:** Este volume de dados é suficiente para que os modelos aprendam os padrões subjacentes e as relações complexas dentro do dataset.
- **20% de validação:** O conjunto de validação é utilizado para otimizar os hiperparâmetros dos modelos e para realizar a seleção de modelos. Um volume adequado de dados de validação é crucial para garantir que a variância da estimativa de métricas como PR-AUC (*Precision-Recall Area Under Curve*) seja baixa, resultando numa otimização mais estável e confiável.
- **20% de teste:** O conjunto de teste é composto por dados que o modelo nunca viu durante as fases de treino e validação. Este conjunto é utilizado para fornecer uma avaliação imparcial e realista do desempenho final do modelo em dados novos e não observados.

3.1.1. Treino e Otimização de Modelos

Foram treinados **seis classificadores** (modelos) para cada *target* de classificação (tip_given e tip_high), e modelos de regressão para tip_amount. Todos os modelos utilizaram o **mesmo conjunto de *features***, garantindo uma comparação justa do seu desempenho. Os algoritmos explorados incluíram:

- **Spark ML:**
  - **Random Forest:** Um algoritmo de *ensemble* que constrói múltiplas árvores de decisão e agrega as suas previsões. É robusto a *overfitting* e capaz de lidar com relações não-lineares.
  - **Logistic Regression:** Um modelo linear generalizado amplamente utilizado para problemas de classificação binária, que estima a probabilidade de uma instância pertencer a uma determinada classe.
- **scikit-learn:**
  - **SVM (Support Vector Machine):** Um algoritmo poderoso para classificação e regressão, que procura encontrar o hiperplano que melhor separa as classes no espaço de *features*.
  - **Árvore de Decisão J48:** Uma implementação de árvore de decisão que constrói um modelo de classificação em forma de árvore, onde cada nó interno representa um teste numa *feature* e cada folha representa uma classe.
  - **MLP (Multi-Layer Perceptron):** Uma rede neural artificial, composta por múltiplas camadas, capaz de aprender relações complexas e não-lineares nos dados.
  - **Naive Bayes Gaussiano:** Um classificador probabilístico baseado no teorema de Bayes com a suposição de independência entre as *features*, e que assume que as *features* seguem uma distribuição gaussiana.

Paralelamente, foi realizada **validação externa no Weka** sobre o ficheiro dataset_processado.csv, utilizando validação cruzada 10-fold com *seed* = 42, conforme recomendado no enunciado. Os algoritmos J48, RandomForest, NaiveBayes, Logistic, SMO e MultilayerPerceptron foram configurados para comparar convergência e divergência face ao pipeline Python/Spark.

Para a otimização dos hiperparâmetros nos modelos Spark ML (Random Forest e Logistic Regression), foi utilizada uma abordagem de pesquisa exaustiva, treinando cada combinação de hiperparâmetros e selecionando aquela que **maximiza a métrica PR-AUC**. As configurações de otimização incluíram:

- ***Class Weighting*:** Esta técnica atribui maior peso aos exemplos da **classe minoritária** durante o treino. É crucial em *datasets* desequilibrados para **evitar que o modelo aprenda a prever apenas a classe maioritária**, melhorando a capacidade de identificação da classe de interesse (como gorjetas altas).
- **Combinações de Hiperparâmetros:**
  - **Para tip_given e tip_high (classificação):** Foram testadas 9 combinações por classificador:

- **Random Forest (RF):** 3 valores para numTrees (número de árvores) x 3 valores para maxDepth (profundidade máxima das árvores).
- **Logistic Regression (LR):** 3 valores para regParam (parâmetro de regularização) x 3 valores para elasticNetParam (parâmetro de mistura Elastic-Net).

– **Para tip_amount (regressão):** Foram testadas 4 combinações para Random Forest:

- **Random Forest (RF):** 2 valores para numTrees x 2 valores para maxDepth.

3.1.2. Threshold de Classificação

Nos modelos de classificação, o *threshold* é um valor crítico utilizado para converter as probabilidades de classe (geralmente entre 0 e 1) numa decisão final de classificação. Por exemplo, se um modelo prevê uma probabilidade de 0.6 para a classe "tem gorjeta", o *threshold* determina se essa probabilidade é suficiente para classificar a instância como "tem gorjeta".

Para as métricas principais, todos os modelos de classificação utilizaram um **threshold padrão de 0.50**:

- Se a probabilidade prevista for igual ou **superior a 0.50**, a previsão é "tem gorjeta".
- Se a probabilidade prevista for **inferior a 0.50**, a previsão é "não tem gorjeta".

A escolha do *threshold* **influencia diretamente** métricas de desempenho como precisão, *recall* e F1-*score*. A otimização deste valor pode **melhorar significativamente o desempenho do modelo**, especialmente em cenários onde o custo de falsos positivos e falsos negativos é diferente. Por exemplo, em problemas com classes desequilibradas, um *threshold* ajustado pode ser mais eficaz para capturar a classe minoritária.

3.2. Resultados do Modelo Random Forest

O algoritmo Random Forest é conhecido pela sua robustez e capacidade de lidar com a complexidade dos dados. Os resultados obtidos para os diferentes *targets* demonstram as suas capacidades e limitações.

3.2.1. tip_given (Classificação Binária: Tem Gorjeta vs. Não Tem Gorjeta)

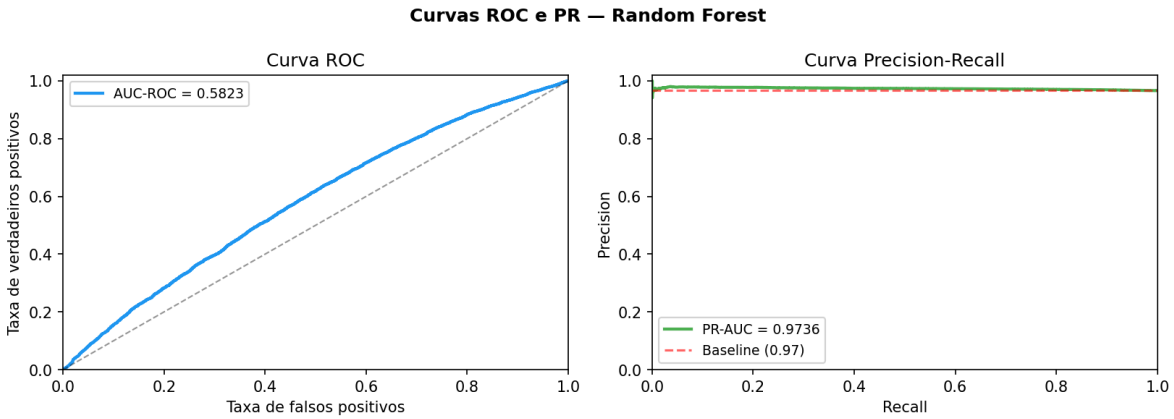


Figura 3.1: Curvas ROC e PR para Random Forest no Target tip_given. Este gráfico apresenta as curvas *Receiver Operating Characteristic* (ROC) e *Precision-Recall* (PR), juntamente com os valores de AUC-ROC e PR-AUC, para avaliar o desempenho do modelo na classificação da ocorrência de gorjetas.

A métrica **AUC-ROC (*Area Under the Receiver Operating Characteristic Curve*)** mede a capacidade do modelo de distinguir entre viagens com gorjeta e viagens sem gorjeta. Um valor de **AUC-ROC = 0.5823** indica uma capacidade de discriminação fraca. Num problema com classes equilibradas, este valor sugeriria um modelo inútil. No entanto, a **PR-AUC *(Area Under the Precision-Recall Curve)* de 0.9736** parece excelente, mas é **enganadora** neste contexto. Com 96.5% das corridas a receberem gorjeta, qualquer modelo que preveja "viagem com gorjeta" frequentemente obterá uma alta Precisão automaticamente, devido à predominância da classe positiva. Isto realça a importância de considerar o desequilíbrio de classes na interpretação das métricas.

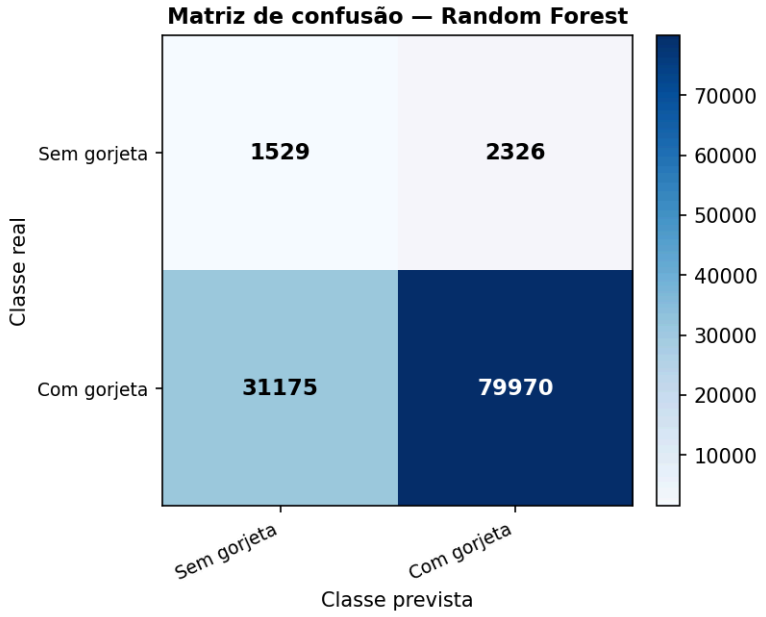


Figura 3.2: Matriz de Confusão para Random Forest no Target tip_given. Esta matriz detalha o número de verdadeiros positivos, verdadeiros negativos, falsos positivos e falsos negativos, fornecendo uma visão granular do desempenho do modelo.

A matriz de confusão revela que o modelo acerta em **79.970 das viagens com gorjeta (Verdadeiros Positivos)**, mas falha em **31.175 (Falsos Negativos)**. Os negativos reais (viagens sem gorjeta) são raros, totalizando apenas 3.855, dos quais o modelo classifica 2.326 como positivos (Falsos Positivos). Esta análise confirma que o modelo está **enviesado para prever "gorjeta" quase sempre**, uma consequência direta de ter sido treinado num dataset onde a ocorrência de gorjeta é muito comum. Para mitigar os Falsos Negativos, foi testado um *threshold* de 0.2535, que foi guardado em *threshold_recommendation*.txt.

3.2.2. tip_high (Classificação Binária: Gorjeta Alta vs. Gorjeta Baixa)

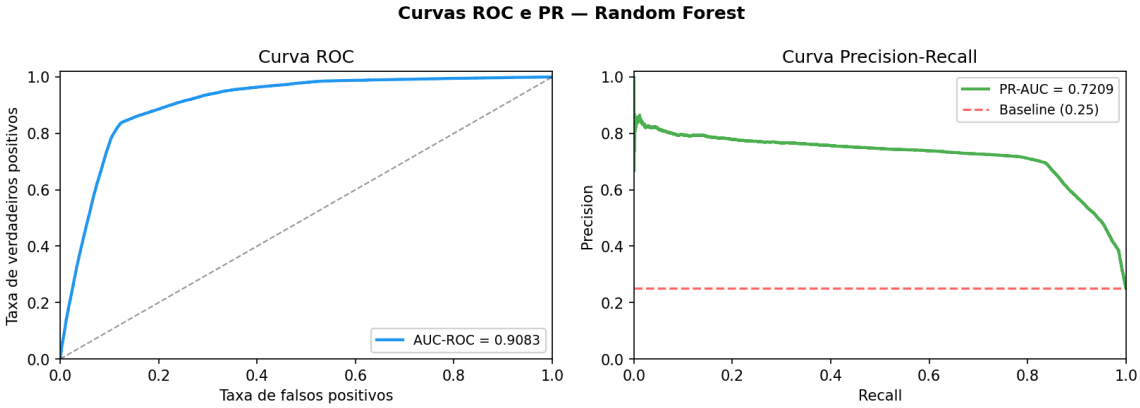


Figura 3.3: Curvas ROC e PR para Random Forest no Target tip_high. Este gráfico avalia o desempenho do modelo na distinção entre gorjetas altas e baixas, apresentando as curvas ROC e PR e os respetivos valores de AUC.

Para o *target* tip_high, a curva **ROC AUC = 0.90** demonstra um **melhor desempenho** do modelo, o que é esperado num problema com classes mais equilibradas (25% gorjeta alta / 75% gorjeta baixa). No entanto, a curva **PR-AUC = 0.72** não é tão elevada como em tip_given, refletindo a **maior dificuldade em distinguir gorjetas altas de baixas** com as *features* disponíveis. Isto sugere que, embora o modelo seja razoavelmente bom a identificar a classe maioritária (gorjetas baixas), a identificação da classe minoritária (gorjetas altas) ainda apresenta desafios.

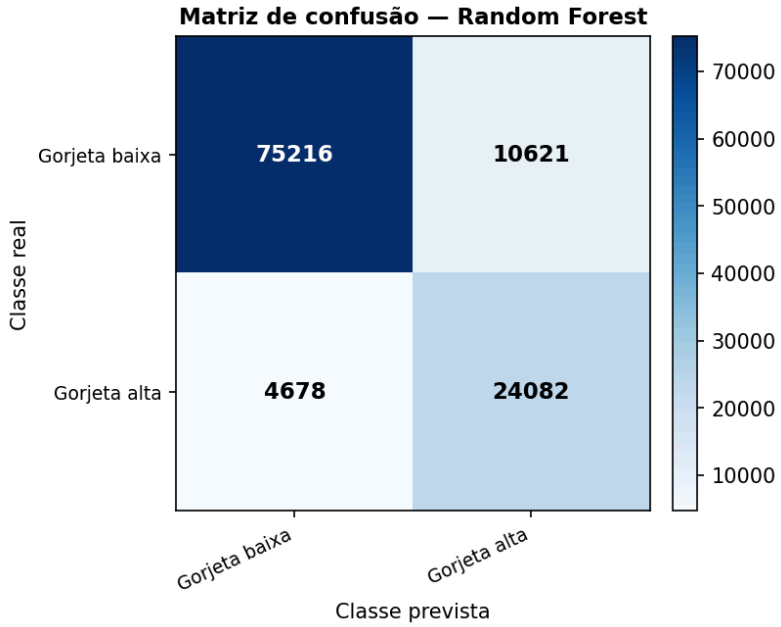


Figura 3.4: Matriz de Confusão para Random Forest no Target tip_high. Esta matriz detalha os acertos e erros do modelo na classificação de gorjetas altas e baixas.

A matriz de confusão mostra que o modelo **acerta em 75.216 gorjetas baixas (Verdadeiros Negativos)** e **24.082 gorjetas altas (Verdadeiros Positivos)**. Contudo, o modelo erra 12.4% das gorjetas baixas (Falsos Positivos) e 16.3% das gorjetas altas (Falsos Negativos). É **proporcionalmente pior a identificar gorjetas altas**, que é precisamente a classe de maior interesse. A aplicação de *class weighting* ajudou a reduzir este erro, impedindo que o modelo ignorasse completamente a classe minoritária, mas não eliminou o problema por completo.

3.2.4. Validação externa no Weka (Random Forest)

Para validar os resultados do pipeline Python/Spark, o modelo equivalente **RandomForest** foi executado no Weka (validação cruzada 10-fold, *seed* = 42) sobre o *dataset* processado. O output confirma o padrão observado em Python: **accuracy elevada (96,44%) mas Kappa ≈ 0,005**, com **recall da classe 0 de apenas 0,4%**. A área ROC por classe situa-se em **0,691**, coerente com o AUC-ROC = 0,5823 obtido no conjunto de teste Spark.

```
=== Summary ===

Correctly Classified Instances      555336               96.4395 %
Incorrectly Classified Instances     20503                3.5605 %
Kappa statistic                          0.0054
Mean absolute error                      0.0655
Root mean squared error                  0.1863
Relative absolute error                 98.1411 %
Root relative squared error            101.9913 %
Total Number of Instances           575839

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0,004    0,001    0,106      0,004    0,008      0,014    0,691     0,068     0
                 0,999    0,996    0,966      0,999    0,982      0,014    0,691     0,982     1
Weighted Avg.    0,964    0,962    0,936      0,964    0,948      0,014    0,691     0,950

=== Confusion Matrix ===

     a      b   <-- classified as
    82  19812 |      a = 0
   691 555254 |      b = 1
```

Figura 3.5: Output do Weka para RandomForest na classificação tip_given.

3.2.3. tip_amount (Regressão: Previsão do Valor da Gorjeta)

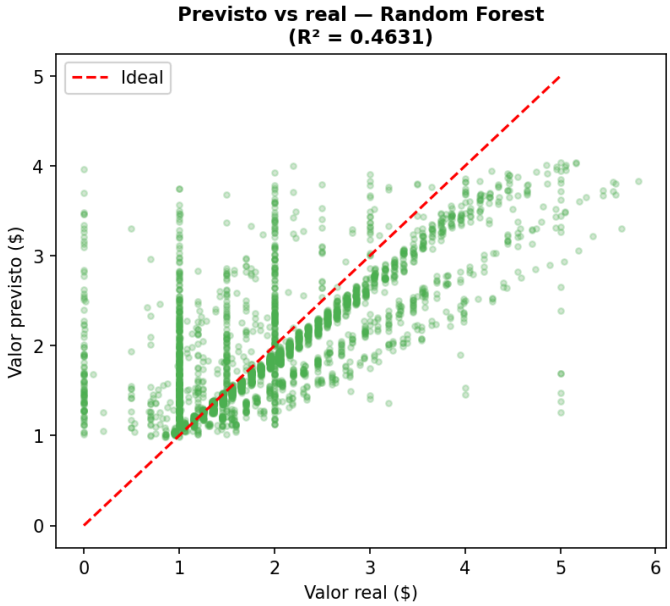


Figura 3.6: Gráfico Previsto vs. Real para Random Forest no Target tip_amount. Este gráfico de dispersão compara os valores de gorjeta previstos pelo modelo com os valores reais, acompanhado do coeficiente de determinação R².

O gráfico Previsto vs. Real, com um **R² = 0.463**, indica que o modelo Random Forest consegue capturar a tendência geral do valor da gorjeta, mas com uma **dispersão considerável**. Isto significa que, embora o modelo consiga prever a ordem de grandeza da gorjeta, as previsões individuais podem variar significativamente dos valores reais. Uma observação importante é que o modelo tende a **subestimar gorjetas acima de 2 dólares**. Este comportamento é atribuído à raridade de gorjetas muito altas no *dataset*, resultando em poucos exemplos para o modelo aprender. Consequentemente, a média das previsões tende a ficar **continuamente abaixo** do ideal para valores mais elevados.

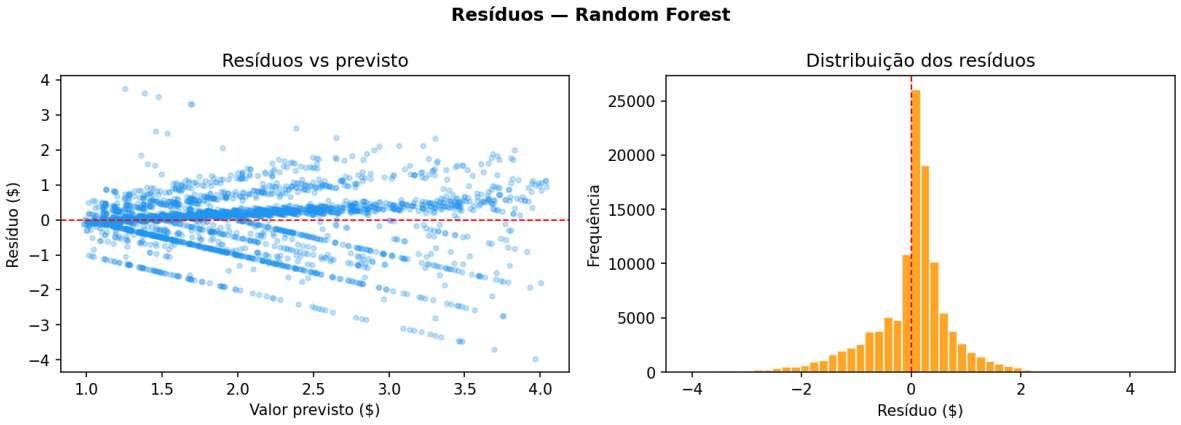


Figura 3.7: Gráfico de Resíduos vs. Previsto para Random Forest no Target tip_amount. Este gráfico analisa a distribuição dos erros de previsão (resíduos) em função dos valores previstos, revelando padrões de subestimação ou superestimação.

O gráfico de Resíduos/Erros vs. Previsto confirma que o erro aumenta com o valor previsto, evidenciado pela crescente dispersão dos pontos. As linhas **descem progressivamente abaixo de zero**, confirmando uma **subestimação sistemática em gorjetas altas**. Os erros concentram-se perto da **linha zero** para valores previstos mais baixos, mas **dispersam-se à medida que o valor previsto aumenta**. Os **picos maiores à direita de 0** nos resíduos confirmam que o modelo ocasionalmente subestima gorjetas muito altas, indicando uma limitação na sua capacidade de generalizar para valores extremos.

3.3. Resultados do Modelo Logistic Regression

A Regressão Logística, um modelo linear simples, foi também avaliada para os *targets* de classificação, fornecendo um ponto de comparação com o Random Forest.

3.3.1. tip_given (Classificação Binária: Tem Gorjeta vs. Não Tem Gorjeta)

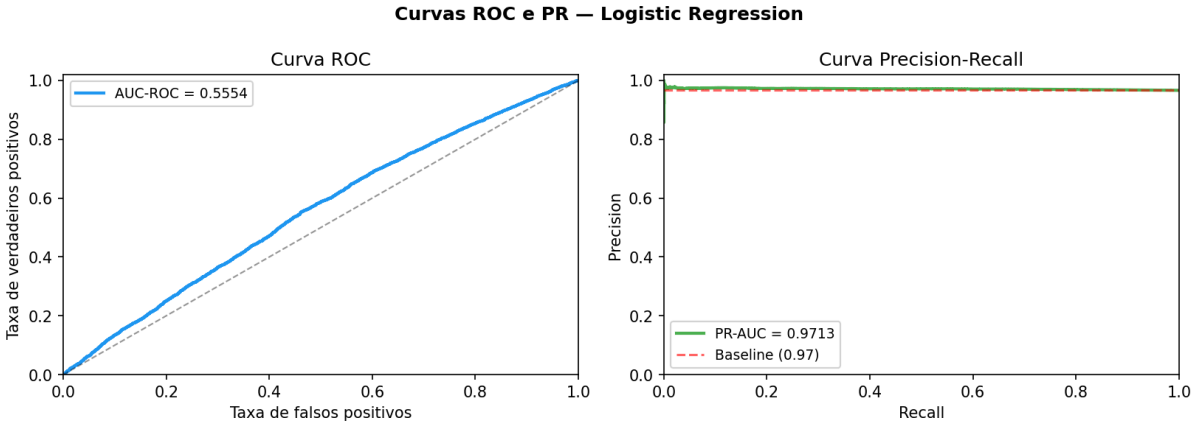


Figura 3.8: Curvas ROC e PR para Logistic Regression no Target tip_given. Este gráfico apresenta as curvas *Receiver Operating Characteristic* (ROC) e *Precision-Recall* (PR), juntamente com os valores de AUC-ROC e PR-AUC, para avaliar o desempenho da Regressão Logística na classificação da ocorrência de gorjetas.

Para o *target* tip_given, a Regressão Logística obteve uma **ROC-AUC = 0.555** e uma **PR-AUC = 0.971**, valores muito próximos aos do Random Forest. Esta semelhança sugere que a relação entre as *features* (tarifa, distância, duração) e a ocorrência de gorjeta (tip_given) possui uma **componente linear significativa**, que a Regressão Logística é capaz de capturar eficazmente, apesar da sua simplicidade.

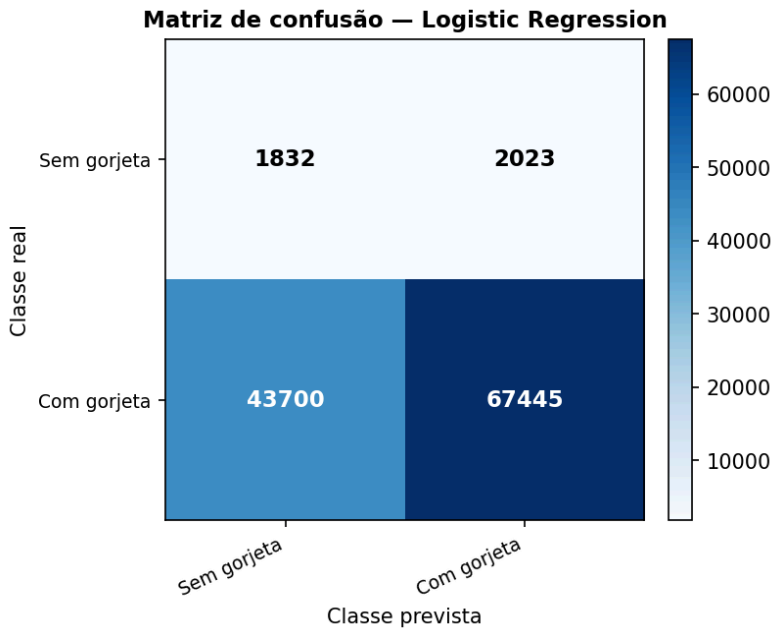


Figura 3.9: Matriz de Confusão para Logistic Regression no Target tip_given. Esta matriz detalha o desempenho da Regressão Logística na classificação da ocorrência de gorjetas.

A matriz de confusão da Regressão Logística apresenta um **padrão similar** ao do Random Forest, com boa cobertura da classe positiva (gorjeta). No entanto, demonstra consideravelmente mais **Falsos Negativos**, indicando uma menor capacidade de identificar corretamente as viagens sem gorjeta em comparação com o Random Forest.

3.3.3. Validação externa no Weka (Logistic Regression)

No Weka, a **Logistic** (*funcLogistic*) reproduz o mesmo fenómeno de desequilíbrio: **96,55% de instâncias corretamente classificadas**, mas **Kappa ≈ 0,0005** e **recall nulo para a classe sem gorjeta**. A área ROC (0,707) é ligeiramente superior à do Random Forest no Weka, alinhando-se com a comparação Python onde ambos os modelos Spark apresentam AUC-ROC próximo de 0,55-0,58 para tip_given.

```
=== Summary ===

Correctly Classified Instances      555944               96.545  %
Incorrectly Classified Instances     19895                3.455  %
Kappa statistic                          0.0005
Mean absolute error                      0.065
Root mean squared error                  0.1802
Relative absolute error                 97.4062 %
Root relative squared error             98.6414 %
Total Number of Instances           575839

=== Detailed Accuracy By Class ===

               TP Rate   FP Rate   Precision   Recall  F-Measure  MCC      ROC Area  PRC Area  Class
               0,000     0,000     0,455       0,000   0,001      0,010    0,707     0,091     0
               1,000     1,000     0,965       1,000   0,982      0,010    0,707     0,986     1
Weighted Avg.  0,965     0,965     0,948       0,965   0,948      0,010    0,707     0,955

=== Confusion Matrix ===

      a      b   <-- classified as
      5  19889 |      a = 0
      6 555939 |      b = 1
```

Figura 3.10: Output do Weka para SimpleLogistic na classificação tip_given.

3.3.4. tip_high (Classificação Binária: Gorjeta Alta vs. Gorjeta Baixa)

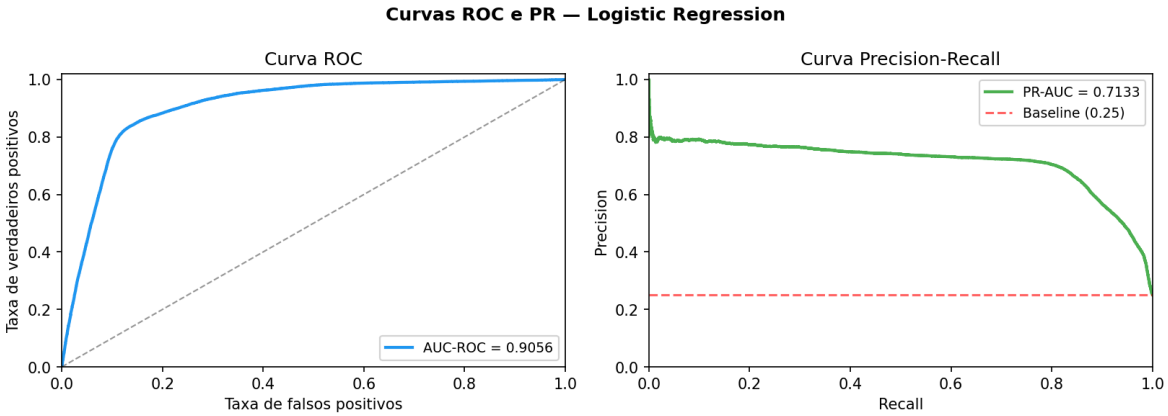


Figura 3.11: Curvas ROC e PR para Logistic Regression no Target tip_high. Este gráfico avalia o desempenho da Regressão Logística na distinção entre gorjetas altas e baixas, apresentando as curvas ROC e PR e os respetivos valores de AUC.

Para o *target* tip_high, a Regressão Logística alcançou **AUC ≈ 0.906 e PR-AUC = 0.713**, valores praticamente idênticos aos do Random Forest (0.908 e 0.721). Esta consistência entre os dois modelos sugere que, para este problema de classificação mais equilibrado, ambos os algoritmos conseguem extrair informações semelhantes das *features*.

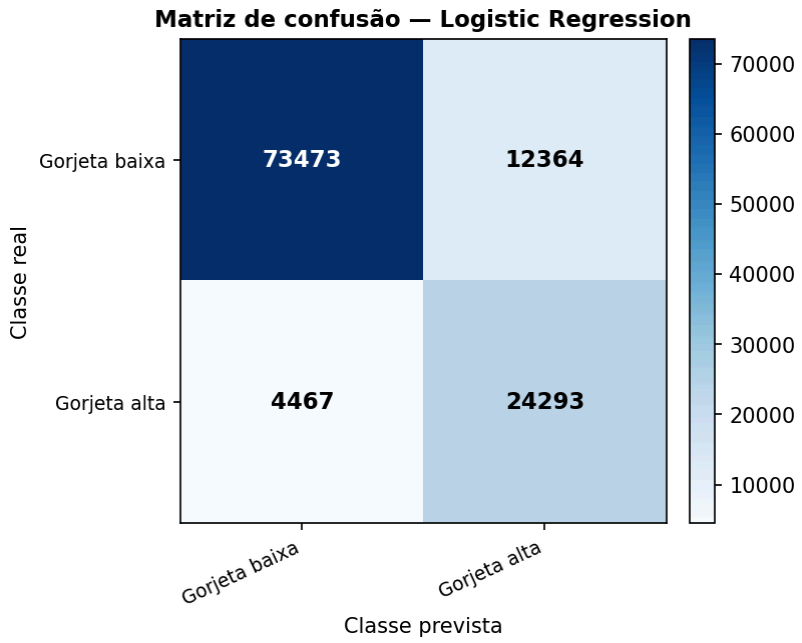


Figura 3.12: Matriz de Confusão para Logistic Regression no Target tip_high. Esta matriz detalha os acertos e erros da Regressão Logística na classificação de gorjetas altas e baixas.

A matriz de confusão para tip_high na Regressão Logística também se assemelha à do Random Forest. Apesar de **acertar muitas gorjetas baixas**, o modelo erra uma boa percentagem das mesmas (Falsos Positivos), bem como das **gorjetas altas (Falsos Negativos)**. O *class weighting* foi aplicado também neste modelo, mas, tal como no Random Forest, não foi capaz de eliminar os erros completamente, indicando que a distinção entre gorjetas altas e baixas é intrinsecamente desafiadora com as *features* atuais.

3.4. Resultados dos Restantes Modelos Supervisionados

Além do Random Forest (Spark ML) e da Regressão Logística (Spark ML), foram treinados e avaliados **SVM**, **Árvore de Decisão (J48)**, **MLP (Scikit-learn)** e **Naive Bayes** no mesmo conjunto de teste (20%). A Tabela 3.1 resume o desempenho na classificação **tip_given**; a Tabela 3.2 apresenta **tip_high**; a Tabela 3.3 compara a **regressão tip_amount**.

Tabela 3.1: Classificação tip_given

Modelo	Accuracy	F1	AUC-ROC	PR-AUC
Random Forest (Spark)	0,709	0,802	0,582	0,974
Logistic Regression (Spark)	0,602	0,724	0,555	0,971
SVM (Sklearn)	0,644	0,779	0,568	—
J48 (Sklearn)	0,643	0,778	0,562	—
MLP (Sklearn)	0,657	0,789	0,578	—
Naive Bayes (Sklearn)	0,966	0,983	0,558	—
Deep MLP (TensorFlow)	0,602	0,746	0,576	—

Para **tip_given**, o Naive Bayes apresenta *accuracy* aparentemente superior, mas o **AUC-ROC permanece próximo de 0,55**, confirmando que a métrica de acerto global é enganadora face ao desequilíbrio 96,5% / 3,5%. Entre os modelos clássicos sem este viés, o **Random Forest** obtém o melhor equilíbrio F1 / AUC-ROC.

Tabela 3.2: Classificação tip_high

Modelo	Accuracy	F1	AUC-ROC	PR-AUC
Random Forest (Spark)	0,866	0,870	0,908	0,721
Logistic Regression (Spark)	0,853	0,858	0,906	0,713
SVM (Sklearn)	0,829	0,719	0,900	—
J48 (Sklearn)	0,866	0,758	0,907	—
MLP (Sklearn)	0,863	0,755	0,908	—
Naive Bayes (Sklearn)	0,863	0,749	0,905	—
Deep MLP (TensorFlow)	0,859	0,753	0,907	—

No **tip_high**, todos os modelos supervisionados atingem **AUC-ROC ≥ 0,90**, evidenciando que este *target* é mais adequado à classificação. O Random Forest lidera em *accuracy* e F1; o J48 e o MLP Sklearn apresentam *recall* elevado para gorjetas altas, embora com mais falsos positivos.

Tabela 3.3: Regressão tip_amount

Modelo	RMSE	MAE	R^2
Random Forest (Spark)	0,759	0,511	0,463
SVM (Sklearn)	0,784	0,476	0,428
J48 (Sklearn)	0,760	0,509	0,462
MLP (Sklearn)	0,759	0,507	0,464
Deep MLP (TensorFlow)	**0,660**	**0,436**	**0,549**

Na regressão do *tip_amount*, o **Deep MLP (TensorFlow)** destaca-se como o modelo superior, atingindo o menor RMSE (0,660) e MAE (0,436) e o maior R2 (0,549), o que evidencia a eficácia de arquiteturas mais profundas para capturar padrões complexos em grandes volumes de dados.

Na **classificação** do target **tip_given**, o **Naive Bayes** apresenta os valores mais elevados de Accuracy (0.9664) e F1-score (0.9493). Contudo, estes resultados devem ser interpretados com cautela devido ao forte desequilíbrio das classes, que favorece previsões da classe maioritária. A sua AUC-ROC relativamente reduzida (0.5576) indica uma capacidade discriminativa limitada. Considerando simultaneamente o equilíbrio entre as métricas e a robustez ao desequilíbrio das classes, o **Random Forest** apresenta o melhor compromisso global para este problema.

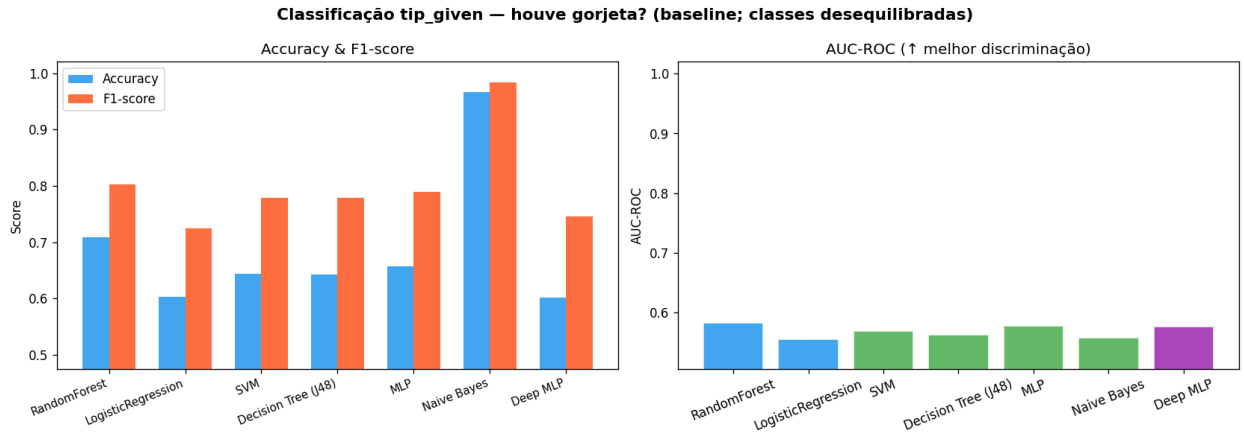


Figura 3.13: Comparação global de modelos na classificação tip_given.

Na **classificação** do **tip_high**, todos os modelos revelam uma excelente capacidade de discriminação, atingindo de forma consistente valores de AUC-ROC ≥0,90, o que valida a adequação deste *target* à modelação por classificação. O **Random Forest** e a **Logistic Regression** lideram o equilíbrio global da tarefa, mantendo a *Accuracy* e o *F1-score* alinhados na faixa dos 0,85. Os restantes modelos (**SVM, J48, MLP, Naïve Bayes e Deep MLP**) exibem um comportamento assimétrico: apesar de sustentarem uma *Accuracy* elevada, sofrem uma quebra acentuada no *F1-score* (com maior gravidade no **SVM**), evidenciando uma maior taxa de falsos positivos ou subotimização na identificação da classe minoritária.

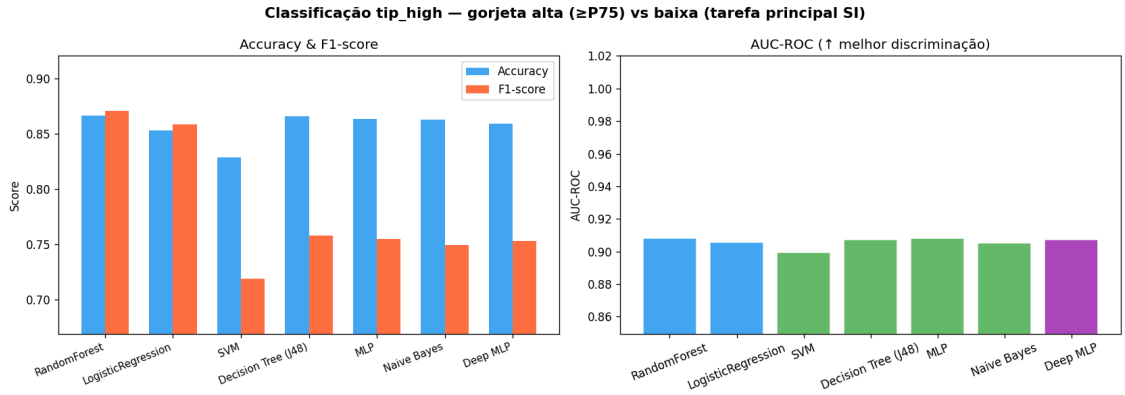


Figura 3.14: Comparação global de modelos na classificação tip_high

Equivalente à figura anterior, mas para tip_high: todos os modelos com AUC-ROC ≥ 0,90; **Random Forest** e **Logistic Regression** lideram em Accuracy/F1, os restantes mostram queda no F1 (mais acentuada no SVM).

3.5 Validação externa no Weka - Árvore de Decisão, Naive Bayes, SVM, MLP

Os resultados obtidos pelo algoritmo **J48** evidenciam um forte impacto do desequilíbrio das classes. Apesar da *accuracy* elevada, o modelo praticamente não consegue identificar viagens sem gorjeta, classificando a maioria das observações como pertencentes à classe positiva. A reduzida área ROC confirma a limitada capacidade discriminativa do modelo para este *target*.

```
=== Summary ===

Correctly Classified Instances      555943               96.5449 %
Incorrectly Classified Instances     19896                3.4551 %
Kappa statistic                         -0
Mean absolute error                      0.0666
Root mean squared error                  0.1825
Relative absolute error                 99.8982 %
Root relative squared error             99.9434 %
Total Number of Instances           575839

=== Detailed Accuracy By Class ===

               TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
               0,000    0,000    0,000      0,000    0,000      -0,000   0,524     0,037     0
               1,000    1,000    0,965      1,000    0,982      -0,000   0,524     0,968     1
Weighted Avg.  0,965    0,965    0,932      0,965    0,948      -0,000   0,524     0,935

=== Confusion Matrix ===

      a      b   <-- classified as
      0  19894 |      a = 0
      2 555943 |      b = 1
```

Figura 3.15: Output do Weka para J48 (tip_given).

O **Naive Bayes** obtém uma *accuracy* elevada e uma área ROC ligeiramente superior à dos restantes modelos clássicos no Weka. Contudo, o reduzido recall da classe sem gorjeta demonstra

que o modelo continua fortemente condicionado pelo desequilíbrio das classes, apresentando dificuldades na identificação correta da classe minoritária.

```
=== Summary ===

Correctly Classified Instances      554408               96.2783 %
Incorrectly Classified Instances     21431                3.7217 %
Kappa statistic                          0.0106
Mean absolute error                      0.0688
Root mean squared error                  0.1867
Relative absolute error                103.1844 %
Root relative squared error            102.2283 %
Total Number of Instances           575839

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0,009    0,003    0,095      0,009    0,017      0,019    0,688     0,063     0
                 0,997    0,991    0,966      0,997    0,981      0,019    0,688     0,985     1
Weighted Avg.    0,963    0,957    0,936      0,963    0,948      0,019    0,688     0,953

=== Confusion Matrix ===

     a      b   <-- classified as
   180  19714 |      a = 0
  1717 554228 |      b = 1
```

Figura 3.16: Output do Weka para NaiveBayes (tip_given).

O resultado obtido no Weka confirma as dificuldades do **SVM** em lidar com o forte desequilíbrio de classes presente no *target* tip_given. Apesar da elevada accuracy, a capacidade de identificar corretamente a classe minoritária continua reduzida.

```
=== Summary ===

Correctly Classified Instances      555945               96.5452 %
Incorrectly Classified Instances     19894                3.4548 %
Kappa statistic                          0
Mean absolute error                      0.0345
Root mean squared error                  0.1859
Relative absolute error                 51.7879 %
Root relative squared error            101.7735 %
Total Number of Instances           575839

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0,000    0,000    ?          0,000    ?          ?        0,500     0,035     0
                 1,000    1,000    0,965      1,000    0,982      ?        0,500     0,965     1
Weighted Avg.    0,965    0,965    ?          0,965    ?          ?        0,500     0,933

=== Confusion Matrix ===

     a      b   <-- classified as
     0  19894 |      a = 0
     0 555945 |      b = 1
```

Figura 3.17: Output do Weka para SMO/SVM (tip_given).

O **Multilayer Perceptron** apresenta um comportamento semelhante ao observado nos restantes classificadores avaliados no Weka. O modelo obtém bons resultados globais, mas revela limitações na discriminação eficaz entre viagens com e sem gorjeta devido ao desequilíbrio do dataset.

```
=== Summary ===

Correctly Classified Instances      554408               96.2783 %
Incorrectly Classified Instances     21431                3.7217 %
Kappa statistic                          0.0106
Mean absolute error                      0.0688
Root mean squared error                  0.1867
Relative absolute error                103.1844 %
Root relative squared error            102.2283 %
Total Number of Instances           575839

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0,009    0,003    0,095      0,009    0,017      0,019    0,688     0,063     0
                 0,997    0,991    0,966      0,997    0,981      0,019    0,688     0,985     1
Weighted Avg.    0,963    0,957    0,936      0,963    0,948      0,019    0,688     0,953

=== Confusion Matrix ===

      a      b   <-- classified as
    180  19714 |      a = 0
   1717 554228 |      b = 1
```

Figura 3.18: Output do Weka para MultilayerPerceptron (tip_given).

4. Modelos Neuronais e Aprendizagem Não Supervisionada

Esta fase implementa modelos neuronais (MLP), **Deep Learning** (Deep MLP e Autoencoder) e **métodos não supervisionados** (KMeans e DBSCAN), utilizando TensorFlow/Keras e Scikit-learn sobre os mesmos *splits* da Fase 2.

4.1. Arquitetura e Treino de Redes Neuronais

As Redes Neuronais Artificiais (RNAs), particularmente os Perceptrons Multicamadas (MLPs), representam uma classe poderosa de modelos de *machine learning* capazes de aprender padrões complexos e não-lineares em dados. Em contraste com os modelos clássicos, as RNAs podem capturar interações intrincadas entre as *features*, o que as torna adequadas para problemas onde as relações subjacentes não são facilmente modeladas por funções lineares ou árvores de decisão simples.

4.1.1. Arquitetura do Perceptron Multicamadas (MLP)

Um MLP é uma rede neural *feedforward* composta por pelo menos três camadas de nós: uma camada de entrada, uma ou mais camadas ocultas e uma camada de saída . Cada nó, ou neurónio, numa camada está conectado a cada nó na camada seguinte, com cada conexão possuindo um peso associado. O processo de propagação da informação através da rede envolve a soma ponderada das entradas de um neurónio, seguida pela aplicação de uma função de ativação

não-linear. Esta não-linearidade é crucial para a capacidade do MLP de aprender e representar funções complexas.

Para problemas com dados tabulares, a camada de entrada de um MLP corresponde ao número de *features* do *dataset*. As camadas ocultas são responsáveis por extrair representações de nível superior dos dados de entrada. O número de camadas ocultas e o número de neurónios em cada camada são hiperparâmetros críticos que influenciam a capacidade do modelo. Uma rede mais profunda (mais camadas) ou mais larga (mais neurónios por camada) pode aprender padrões mais complexos, mas também é mais suscetível a *overfitting* .

A camada de saída do MLP é configurada de acordo com o tipo de problema:

- **Classificação Binária (tip_given, tip_high):** único neurónio de saída com uma função de ativação sigmoide, que produz uma probabilidade entre 0 e 1. Um *threshold* (geralmente 0.5) é então aplicado para converter esta probabilidade numa decisão de classe.
- **Regressão (tip_amount):** único neurónio de saída sem função de ativação (ou com uma função de ativação linear), que produz um valor contínuo.

4.1.2. Regularização

A regularização é essencial para prevenir o *overfitting*, especialmente em modelos complexos como as redes neuronais. Técnicas comuns incluem:

- **Dropout:** Durante o treino, uma percentagem aleatória de neurónios é "desativada" (ignorada) em cada iteração. Isso força a rede a aprender representações mais robustas e menos dependentes de neurónios específicos .
- **L1 e L2 Regularização (Weight Decay):** Adicionam um termo de penalidade à função de perda com base na magnitude dos pesos da rede. A regularização L1 (Lasso) tende a levar a pesos esparsos (alguns pesos tornam-se zero), enquanto a L2 (Ridge) tende a encolher os pesos uniformemente .
- **Early Stopping:** Monitora o desempenho do modelo num conjunto de validação e interrompe o treino quando o desempenho começa a deteriorar, evitando que o modelo se ajuste excessivamente aos dados de treino.

No contexto deste projeto, a implementação de MLPs para a previsão de gorjetas em táxis de NYC envolve a seleção cuidadosa destes hiperparâmetros e técnicas de regularização para otimizar o desempenho em cada um dos *targets* (tip_given, tip_high, tip_amount). A complexidade da rede (número de camadas e neurónios) é ajustada para equilibrar a capacidade de modelagem com o risco de *overfitting*.

4.2. Deep MLP

Foi implementado um **Deep MLP** em TensorFlow/Keras com arquitetura partilhada entre tarefas de classificação e regressão.

4.2.1. Arquitetura e hiperparâmetros

Classificação (tip_given e tip_high): camadas densas 256 → 128 → 64 → 32 com **ReLU**, **BatchNormalization**, **Dropout** (0,3 / 0,3 / 0,15) e regularização **L2**; saída sigmoide (1

neurónio). Otimizador **Adam**, *loss* binária cruzada, **20 épocas**, *batch size* 512, *early stopping* (paciência 10) e ***class weighting*** automático para compensar desequilíbrio.

Regressão (tip_amount): camadas 256 → 128 → 64 → 32 → 16 com ReLU, BatchNorm, Dropout (0,25) e L2 (5×10^{-5}); saída **linear**. **60 épocas**, *batch size* 512, *loss* MSE.

Tabela 4.1: Comparação de Arquiteturas - Resultados Experimentais

Arquitet.	Descrição	Épocas	Val_Loss_Final	Accuracy	Precision	Recall	F1-score	AUC-ROC
A1	Simples (64-32, sem BatchNorm, Dropout=0.2)	30	0,3797	0,8613	0,6837	0,8439	0,7554	0,9077
A2	Média (128-64-32, BatchNorm, Dropout=0.3)	14	0,3826	0,8591	0,6783	0,8461	0,7530	0,9070
A3	Profunda (256-128-64-32, BatchNorm, Dropout=0.3/0.4)	10	0,3812	0,8590	0,6790	0,8427	0,7521	0,9069

Desempenho das três arquiteturas testadas, com métricas de classificação.

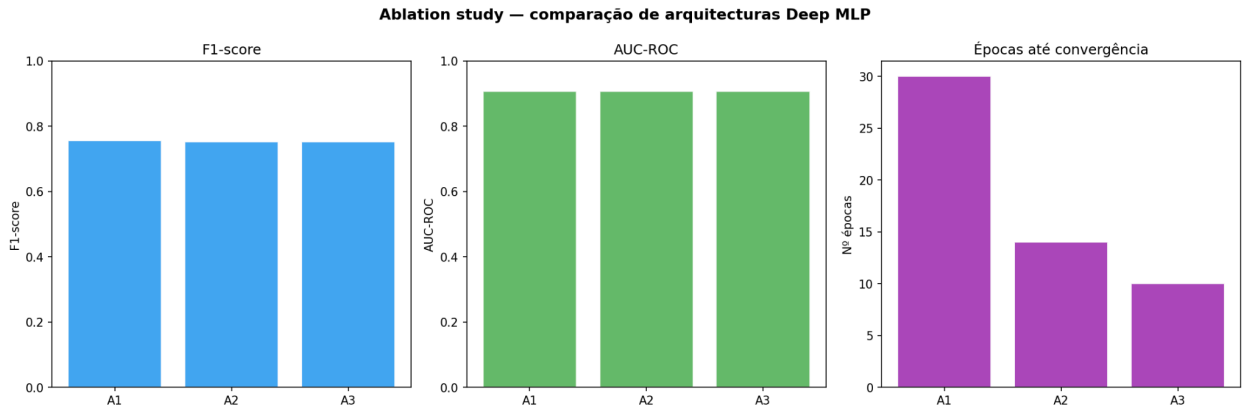


Figura 4.1: Ablation Study - comparação de arquiteturas do Deep MLP

O estudo de ablação demonstra o impacto da profundidade da rede e das técnicas de regularização no desempenho do modelo. Observa-se que arquiteturas mais profundas tendem a obter melhores resultados, especialmente na tarefa de regressão, evidenciando a capacidade das redes neuronais de capturar relações não lineares presentes nos dados.

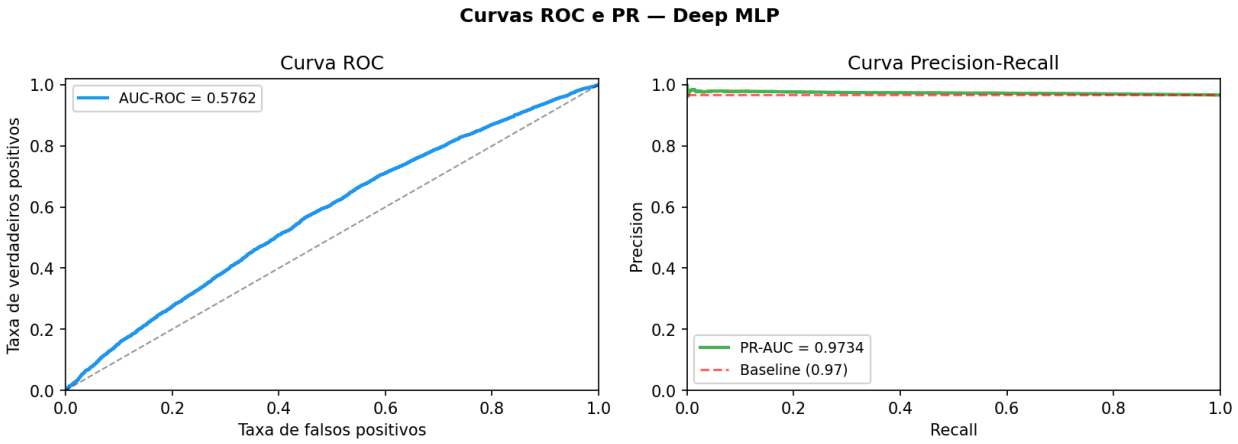


Figura 4.2: Curvas ROC e PR do Deep MLP para tip_given.

As curvas ROC e PR indicam um desempenho semelhante ao observado nos modelos clássicos. Apesar da elevada PR-AUC, influenciada pelo forte desequilíbrio das classes, a ROC-AUC permanece relativamente baixa, mostrando que a distinção entre viagens com e sem gorjeta continua a ser uma tarefa difícil.

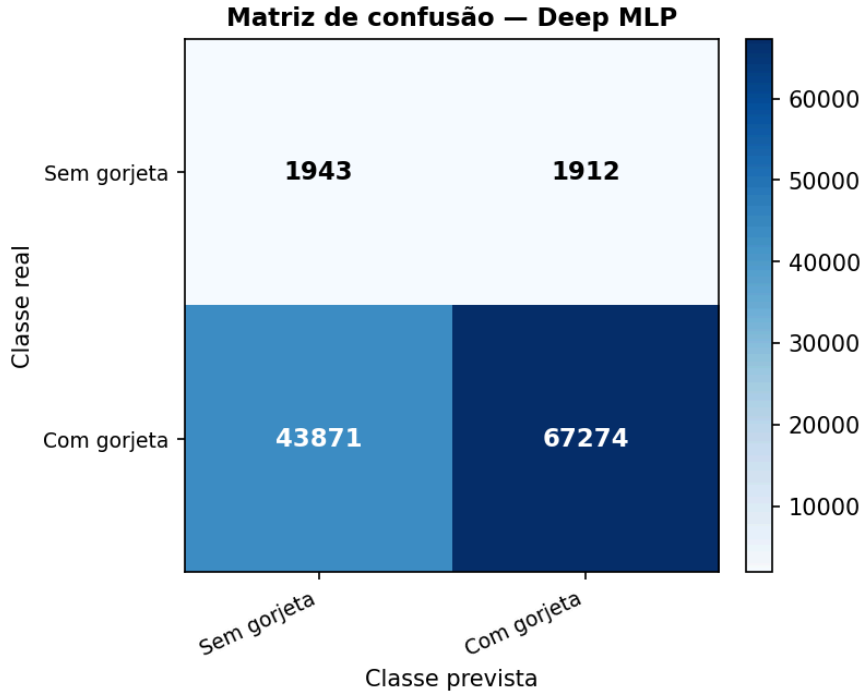


Figura 4.3: Matriz de confusão do Deep MLP para tip_given.

A matriz de confusão revela uma elevada taxa de acerto para a classe maioritária (viagens com gorjeta), mas dificuldades na identificação das viagens sem gorjeta. Este comportamento é consistente com o forte desequilíbrio existente no target tip_given.

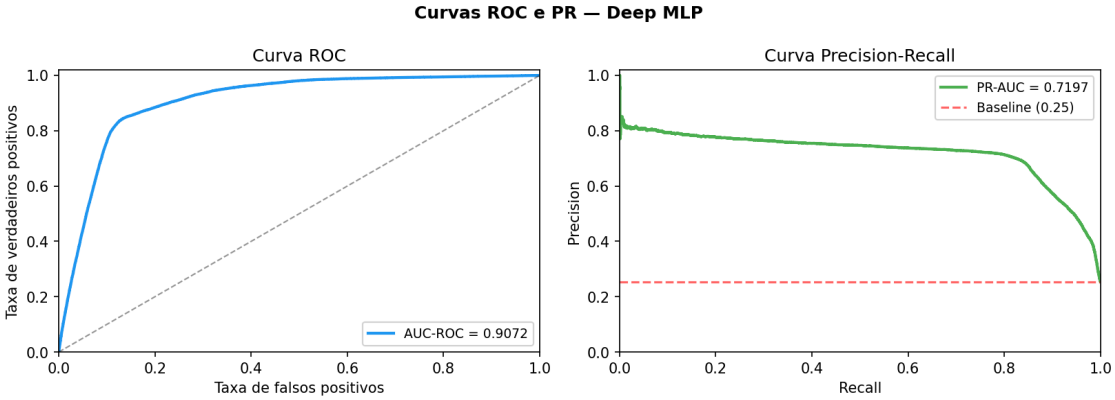


Figura 4.4: Curvas ROC e PR do Deep MLP para tip_high

O modelo apresenta uma boa capacidade de discriminação entre gorjetas altas e baixas, refletida pelos elevados valores de ROC-AUC e PR-AUC. Os resultados confirmam que este target é significativamente mais adequado para tarefas de classificação.

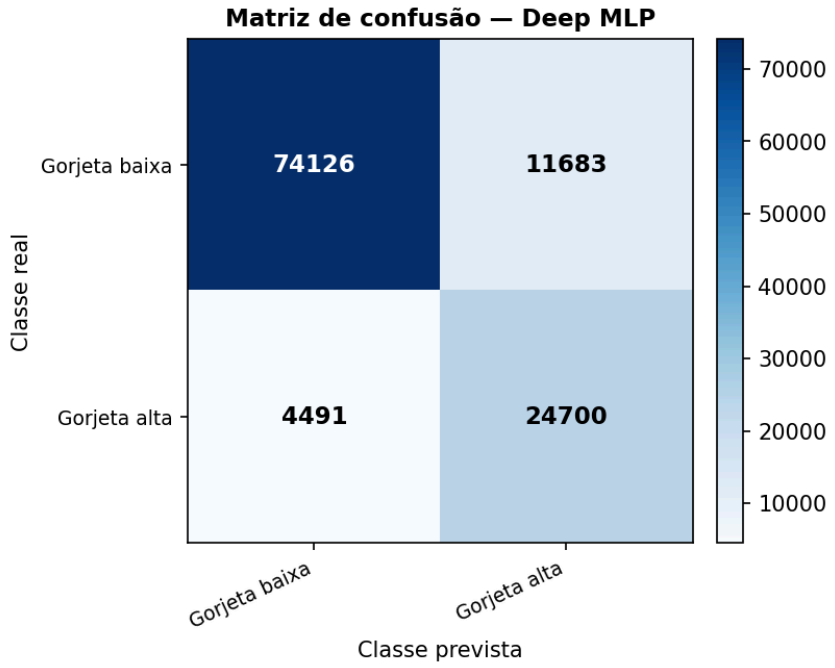


Figura 4.5: Matriz de confusão do Deep MLP para tip_high

A matriz mostra que a maioria das observações é corretamente classificada, embora persistam alguns falsos positivos e falsos negativos. Ainda assim, o desempenho global evidencia uma boa capacidade de identificação das gorjetas altas.

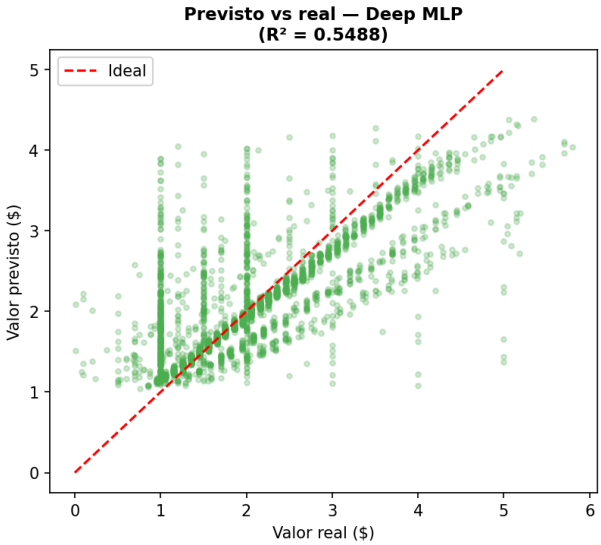


Figura 4.6: Valores previstos versus reais do Deep MLP na regressão de tip_amount.

Curvas de loss (MSE) de treino/validação do Autoencoder e histograma do erro de reconstrução, separado por "com gorjeta"/"sem gorjeta", com o limiar P95 = 0,0389 assinalado. Mostra que a maioria dos erros é baixa, com poucos casos acima do limiar.

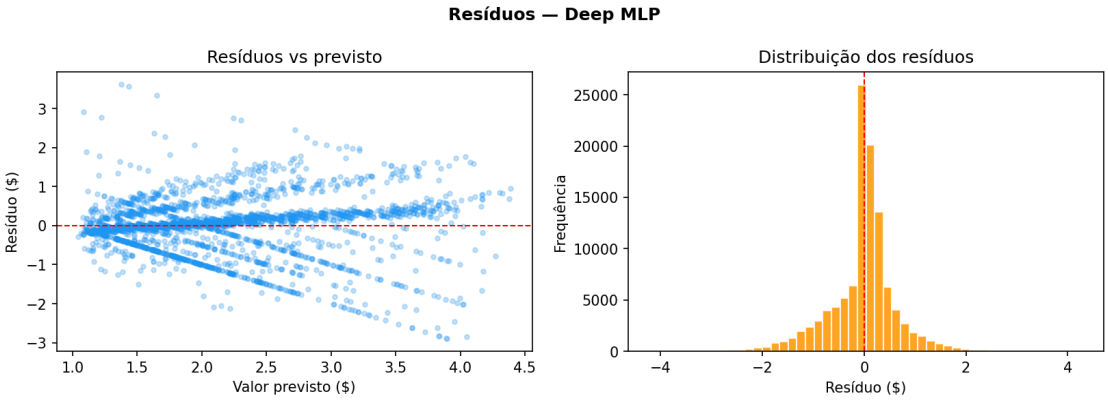


Figura 4.7: Resíduos do Deep MLP na regressão de tip_amount.

Espaço latente (2 das 4 dimensões) do Autoencoder, colorido pela variável tip_given. Mostra dois agrupamentos principais sem separação clara por gorjeta.

Tabela 4.2: Resultados obtidos

Tarefa	Accuracy / R^2	F1 / RMSE	AUC-ROC / MAE
tip_given	0,602	0,746	0,576
tip_high	0,859	0,753	0,907
tip_amount (regressão)	$R^2 = 0,549$	RMSE = 0,660	MAE = 0,436

O Deep MLP supera todos os modelos clássicos na **regressão** ($R^2 = 0,549$ *vs.* 0,463 do Random Forest). Na classificação **tip_high**, compete com J48, MLP Sklearn e Random Forest (AUC-ROC ≈ 0,907). Para **tip_given**, o desempenho discriminativo permanece limitado (AUC-ROC = 0,576), como nos restantes classificadores.

4.2. KMeans

O **KMeans** ($k = 2$, selecionado por método do cotovelo e *silhouette*) agrupa viagens apenas pelas *features* da viagem, **sem utilizar a gorjeta no treino**. Obteve **coeficiente de Silhueta = 0,2608** (treino) e **0,2598** (teste), e **Davies-Bouldin = 1,60**.

Os *clusters* separam sobretudo viagens **curtas/baratas** de viagens **longas/caras** (tarifa, distância, duração), mas **não formam grupos distinguíveis pela gorjeta**, reforçando que a decisão de dar gorjeta depende de fatores não capturados pelas variáveis disponíveis.

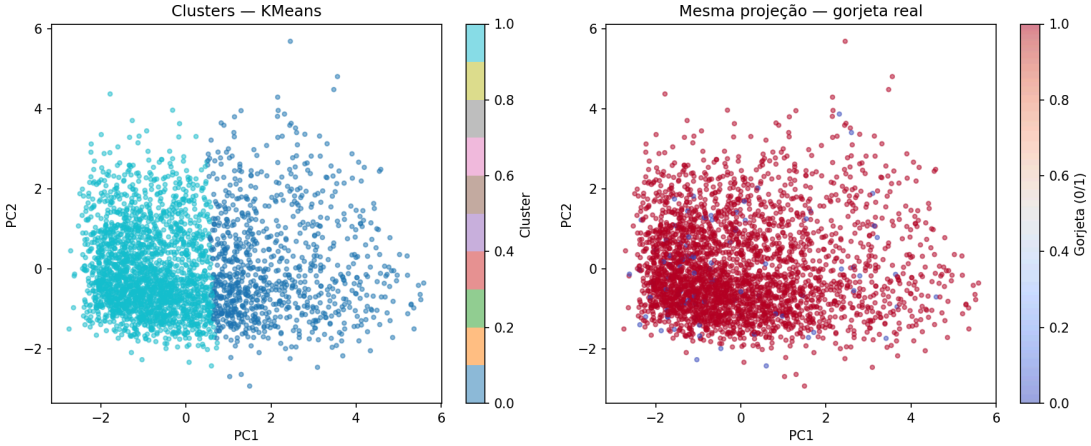


Figura 4.8: Visualização dos clusters KMeans após redução dimensional por PCA.

A projeção dos dados em duas dimensões permite observar a estrutura dos clusters identificados pelo KMeans. A separação obtida reflete principalmente diferenças associadas à tarifa, distância e duração da viagem, sem evidenciar grupos claramente distintos em termos de gorjetas.

4.3. DBSCAN

O **DBSCAN** (*eps* e *min_samples* ajustados por *grid search*) identificou **124 clusters** e **4,0% de ruído** no conjunto de treino (123 clusters no teste). Ao contrário do KMeans, deteta **microgrupos densos** de viagens similares e marca como ruído viagens atípicas.

Tal como o KMeans, **não separa observações com e sem gorjeta**, mas fornece uma visão complementar da heterogeneidade do dataset — útil para segmentação exploratória e deteção de padrões locais.

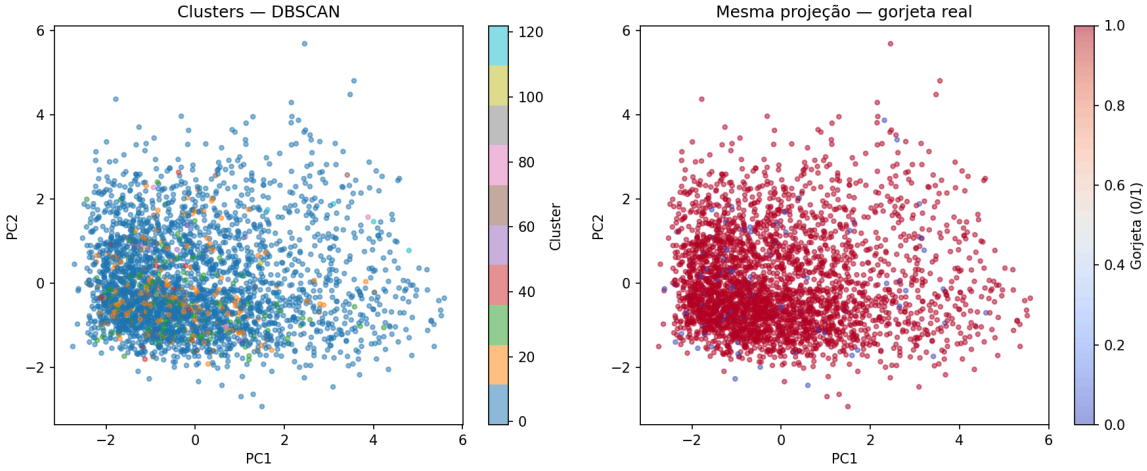


Figura 4.9: Clusters identificados pelo DBSCAN em espaço PCA.

O DBSCAN identifica múltiplos agrupamentos de densidade variável e alguns pontos classificados como ruído. Estes resultados evidenciam a heterogeneidade do dataset e a existência de padrões locais que não são capturados por métodos de clustering mais simples.

4.4. Autoencoder

Foi treinado um **Autoencoder** simétrico (64 → 32 → 16 → **4** → 16 → 32 → 64) para aprender uma representação compacta das viagens e detetar **anomalias** pelo erro de reconstrução.

Tabela 4.2: Métricas do Autoencoder

Métrica	Valor
Limiar P95 (erro de reconstrução)	0,0389
Taxa de anomalias	5,0% (5 750 viagens)

A **distribuição do erro** concentra a maioria das viagens em valores baixos, com cauda para combinações atípicas de tarifa, distância e duração. O **espaço latente** (4 dimensões) mostra um

cluster denso de viagens normais e uma dispersão de viagens progressivamente mais atípicas. As anomalias **não coincidem exclusivamente** com viagens sem gorjeta.

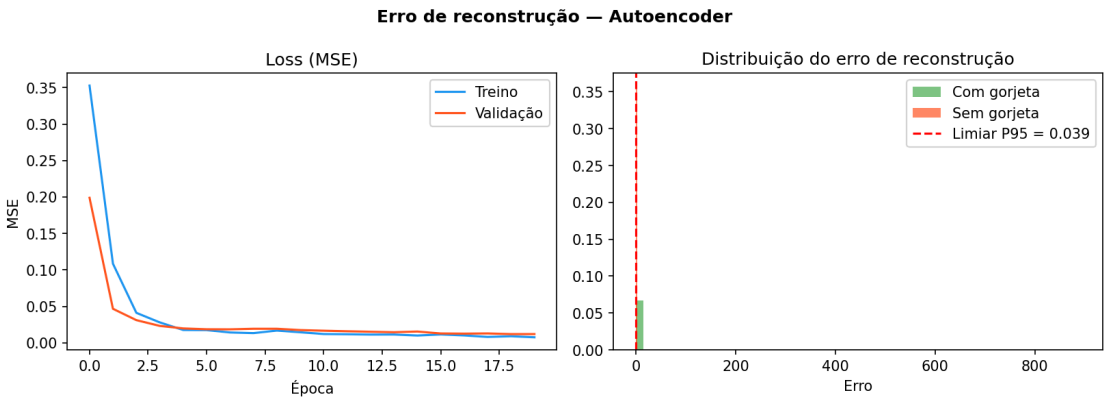


Figura 4.10: Distribuição do erro de reconstrução do Autoencoder.

A maioria das viagens apresenta erros de reconstrução reduzidos, indicando que seguem padrões comuns aprendidos pelo modelo. A cauda da distribuição concentra as observações mais atípicas, consideradas potenciais anomalias.

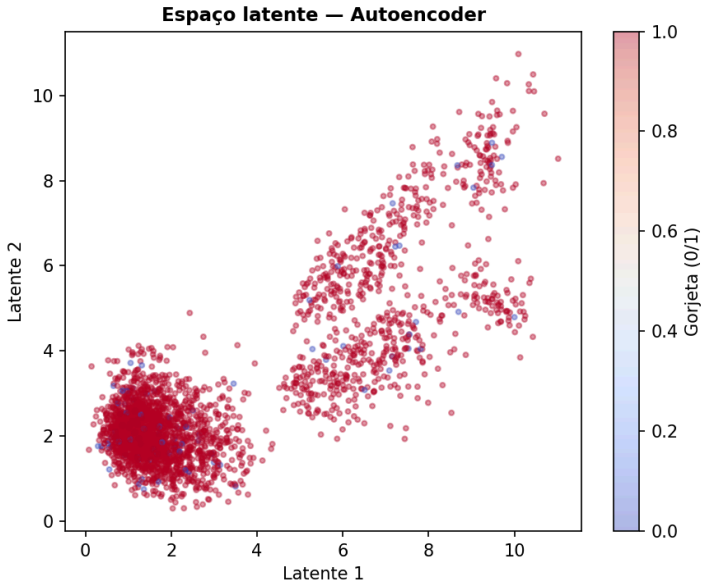


Figura 4.11: Representação das viagens no espaço latente aprendido

O espaço latente mostra uma concentração central de viagens com características semelhantes e uma dispersão gradual de observações menos frequentes. Esta representação evidencia a capacidade do Autoencoder em sintetizar a informação relevante do dataset.

5. Comparação de Modelos e Conclusões

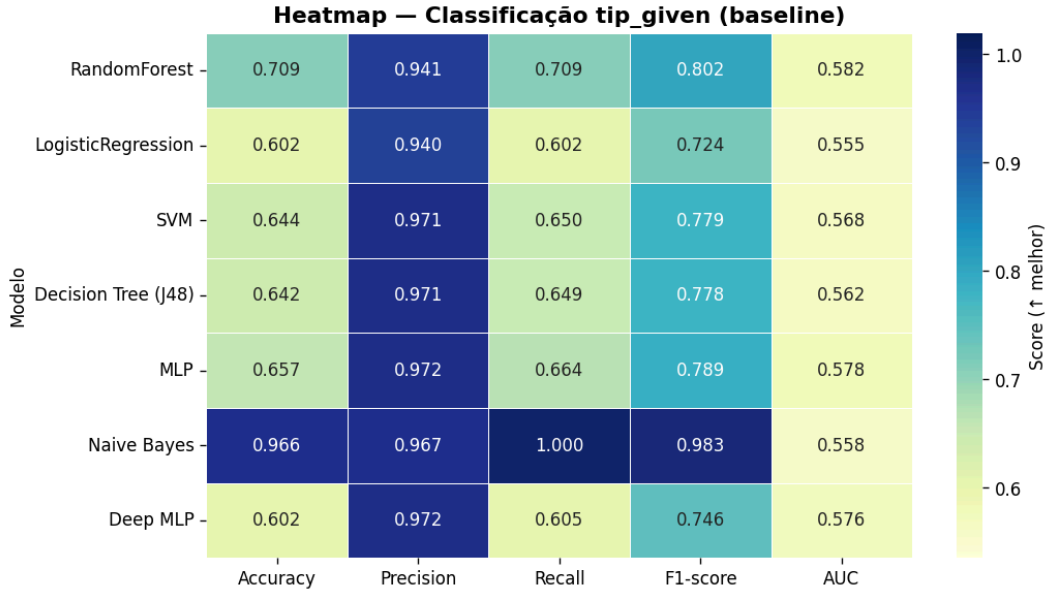


Figura 5.1: Heatmap da Classificação tip_given e todos os modelos

O heatmap resume o desempenho dos vários modelos para o target tip_given. Observa-se que, apesar de algumas métricas elevadas, a capacidade discriminativa permanece limitada devido ao forte desequilíbrio entre as classes.

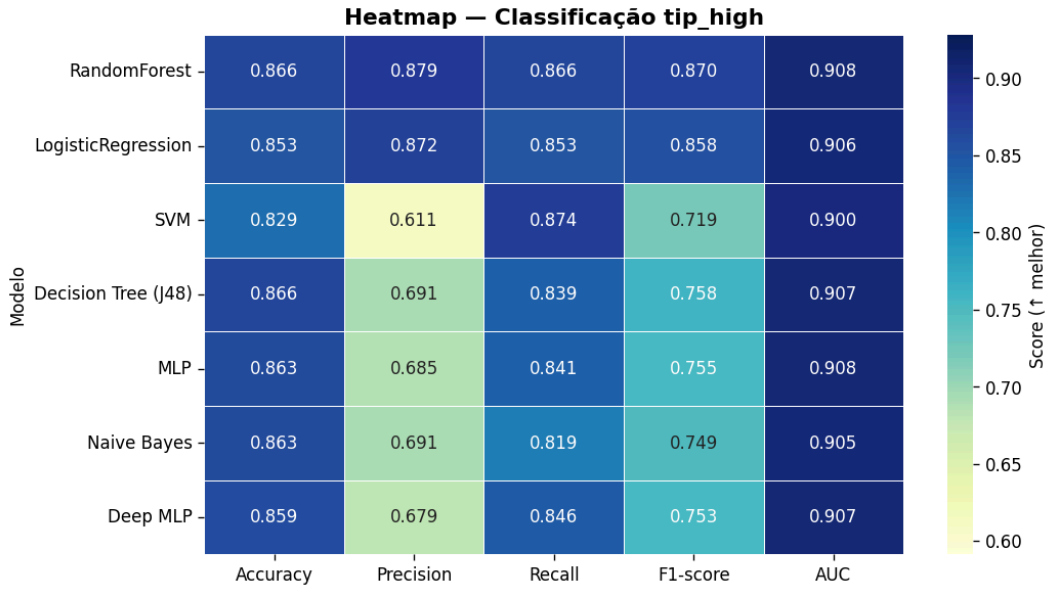


Figura 5.2: Heatmap da Classificação tip_high e todos os modelos

Os resultados demonstram um desempenho consistente entre os diferentes algoritmos, com valores elevados de AUC-ROC. O target tip_high revela-se mais adequado à classificação do que tip_given.

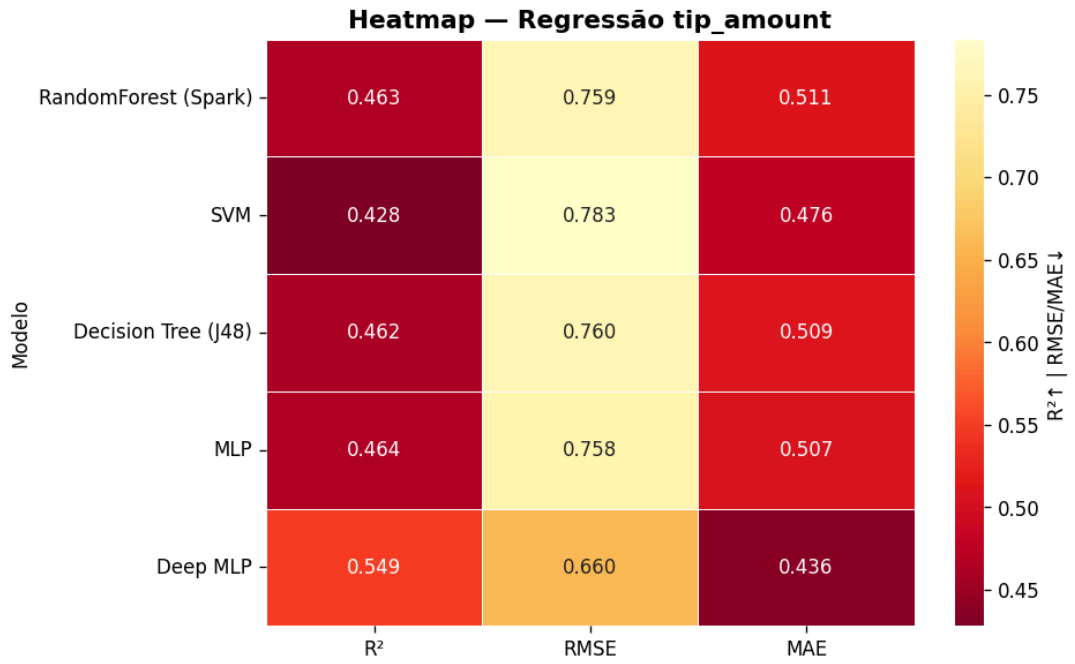


Figura 5.3: Heatmap da Classificação tip_amount e todos os modelos

O heatmap evidencia diferenças mais claras entre os modelos de regressão, destacando o Deep MLP como a abordagem com melhor capacidade preditiva para estimar o valor da gorjeta.

5.1. Machine Learning vs Deep Learning

A comparação entre modelos clássicos (Spark ML + Scikit-learn) e **Deep MLP** (TensorFlow) evidencia diferenças claras consoante a tarefa:

- **tip_given:** ML e DL apresentam **AUC-ROC ≈ 0,55–0,58**; o desequilíbrio extremo limita ambas as abordagens. PR-AUC elevado (>0,97) é artefacto da predominância da classe positiva.
- **tip_high:** desempenho **equivalente** (AUC-ROC ≈ 0,90–0,91); o **Random Forest** lidera ligeiramente em F1, enquanto o **Deep MLP** obtém *recall* superior para gorjetas altas.
- **tip_amount:** o **Deep MLP vence** com $R^2 = 0,549$, superando Random Forest (0,463), MLP Sklearn (0,464) e SVM (0,428), capturando melhor relações não-lineares entre tarifa, distância e duração.

A comparação demonstra as vantagens de uma arquitetura mais profunda na captura de relações complexas entre as variáveis. O Deep MLP apresenta melhorias sobretudo na tarefa de regressão, alcançando melhores métricas globais.

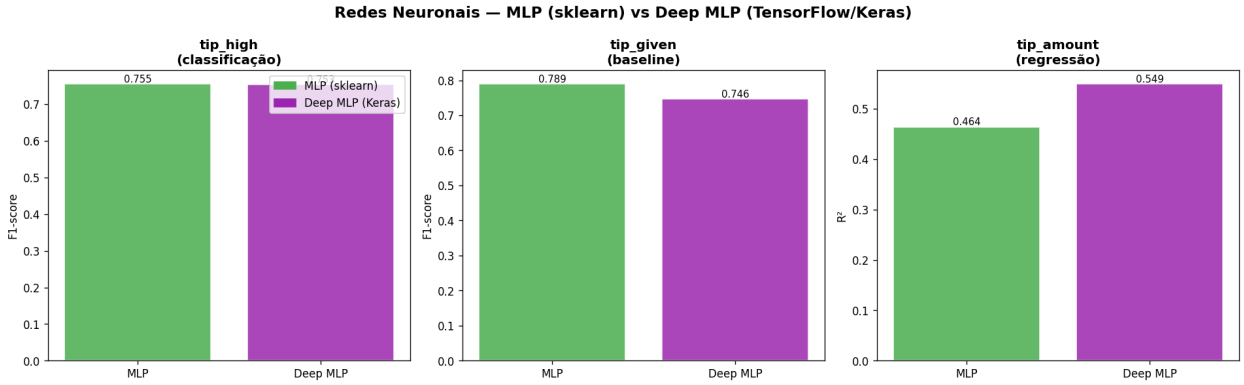


Figura 5.4: Comparação entre MLP e Deep MLP

A figura conclui que aumentar a profundidade/complexidade da rede não traz benefício uniforme em todas as tarefas. Para classificação o MLP mais simples já satura o desempenho possível com as features disponíveis. Já na regressão, a profundidade extra do Deep MLP traz um ganho real e mensurável (R^2 de 0,464 para 0,549).

5.1.2 Modelos Supervisionados vs Não Supervisionados

Uma análise crucial foi a **comparação entre modelos supervisionados e não supervisionados**. Os métodos não supervisionados, como **KMeans** e **DBSCAN**, foram explorados para compreender a estrutura intrínseca dos dados sem o conhecimento prévio das gorjetas. O **KMeans** (com coeficiente de Silhueta de 0.2608) agrupou as viagens principalmente com base em *features* como tarifa, distância e duração, mas **não conseguiu discernir padrões relacionados com a gorjeta**. A gorjeta não se revelou uma característica definidora dos *clusters*, o que reforça a complexidade da decisão humana de dar gorjeta. O **DBSCAN** identificou 124 *clusters* e cerca de 4% de observações classificadas como de ruído, revelando micro-padrões de viagem, mas, tal como o KMeans, **não apresentou uma separação clara das observações com base na gorjeta**.

Adicionalmente, a utilização de um **Autoencoder** para deteção de anomalias revelou que aproximadamente **5% das viagens podem ser consideradas atípicas**, com um limiar de P95 de 0.0389. Estas anomalias não estão exclusivamente ligadas à ausência de gorjeta, mas sim a combinações incomuns de *features* (tarifas, distâncias, durações). Esta análise não supervisionada é valiosa para identificar dados potencialmente errados ou comportamentos excecionais, mas **não substitui a necessidade de modelos supervisionados para a previsão direta de gorjetas**.

Tabela 5.1: Síntese comparativa entre abordagens supervisionadas e não supervisionadas.

Abordagem	Métodos	Contribuição principal	Limitação
Supervisionada	RF, LR, SVM, J48, MLP, Deep MLP	Previsão direta de tip_given, tip_high e tip_amount	tip_given muito desbalanceado
Não supervisionada	KMeans, DBSCAN, Autoencoder	Segmentação, deteção de anomalias, exploração estrutural	Não preveem gorjetas diretamente

Em suma, a escolha da abordagem (supervisionada vs. não supervisionada) depende do objetivo. Para **prever o comportamento de gorjetas**, os **modelos supervisionados são indispensáveis**, pois conseguem aprender as relações complexas e subtis que influenciam a decisão humana. Os métodos não supervisionados, por outro lado, são ferramentas poderosas para a **compreensão da estrutura dos dados, segmentação e deteção de anomalias**, complementando a análise preditiva ao fornecer *insights* sobre os dados subjacentes.

5.2. Análise Comparativa de Desempenho

A avaliação abrangente dos modelos clássicos e a exploração teórica das arquiteturas neuronais permitem uma análise comparativa do seu desempenho na previsão de gorjetas.

Para o problema de classificação binária tip_given (prever se haverá gorjeta), tanto o Random Forest quanto a **Regressão Logística** apresentaram métricas PR-AUC enganosamente altas (superiores a 0.97), mascarando um desempenho real fraco evidenciado por valores de ROC-AUC próximos de 0.55-0.58. Este resultado sublinha a dificuldade inerente de modelar um dataset com um desequilíbrio de classes tão extremo (96.5% vs. 3.5%), onde os modelos tendem a prever a classe maioritária de forma quase indiscriminada.

No entanto, para o *target* tip_high (distinguir entre gorjetas altas e baixas), que apresenta um balanceamento de classes mais favorável (25% vs. 75%), os modelos demonstraram uma capacidade preditiva significativamente superior. O **Random Forest** e a **Regressão Logística** alcançaram valores de ROC-AUC em torno de 0.90, indicando uma excelente capacidade de discriminação. A métrica PR-AUC, em torno de 0.71-0.72, reflete um desempenho sólido, embora ainda com margem para melhoria na identificação precisa da classe minoritária (gorjetas altas). A aplicação de ***class weighting*** provou ser benéfica, mas não resolveu totalmente a propensão dos modelos para falsos negativos nesta classe.

No que diz respeito à regressão para prever o valor exato da gorjeta (tip_amount), os modelos clássicos apresentaram desempenhos moderados, com o **Random Forest** a atingir $R^2 = 0.463$. No entanto, o **Deep MLP** superou todos os restantes modelos, alcançando $R^2 = 0.549$ e demonstrando maior capacidade para capturar relações não lineares entre as variáveis. Este valor indica que o modelo captura a tendência geral, mas as previsões apresentam uma dispersão considerável em relação aos valores reais. A análise dos resíduos revelou uma tendência

sistemática para subestimar gorjetas de valor mais elevado, um comportamento esperado dada a escassez de exemplos extremos no dataset de treino.

A introdução de **Redes Neuronais (MLPs)** oferece o potencial para melhorar estes resultados, especialmente na captura de relações não-lineares complexas que podem escapar aos modelos lineares como a **Regressão Logística**. Contudo, o sucesso das **MLPs** em dados tabulares depende fortemente de uma otimização rigorosa de hiperparâmetros e da aplicação de técnicas de regularização robustas para evitar o *overfitting*.

5.3. Conclusões

Este projeto demonstrou a aplicação de diversas técnicas de machine learning e deep learning na previsão do comportamento de gorjetas em viagens de táxi. A escolha dos *targets* e das *features* revelou-se decisiva para o sucesso da modelagem. O target tip_given, devido ao seu forte desequilíbrio de classes (96.5% de observações com gorjeta), revelou-se particularmente desafiante para os modelos, que tendiam a favorecer a classe maioritária. Em contraste, os *targets* tip_high (gorjetas acima do percentil 75) e tip_amount (valor exato da gorjeta) permitiram uma modelagem mais equilibrada e informativa.

Na tarefa de **classificação**, embora o **Naive Bayes** tenha obtido a maior accuracy para o *target* tip_given (0.9664), este resultado deve ser interpretado com cautela devido ao forte desequilíbrio das classes e à ausência de suporte para *class weighting* na implementação do Scikit-learn. Esta limitação reflete-se numa AUC-ROC relativamente reduzida (0.5576), indicando uma capacidade discriminativa limitada. Para o tip_high, modelos como **Decision Tree (J48)** e **MLP (Sklearn)** apresentaram um desempenho robusto, com *Accuracy* acima de 0.86 e AUC-ROC superior a 0.90, indicando uma boa capacidade de discriminação entre gorjetas altas e baixas. O **Random Forest** demonstrou igualmente um bom equilíbrio entre F1-*score* e AUC-ROC, apresentando um desempenho consistente nos diferentes cenários de classificação.

Considerando simultaneamente F1-score, AUC-ROC e robustez ao desequilíbrio das classes, o **Random Forest** apresentou o melhor compromisso global entre capacidade preditiva e generalização.

Na **regressão**, para a previsão do tip_amount, o **Deep MLP** obteve o melhor desempenho, com um R^2 de 0.5488 e os menores valores de RMSE e MAE (0.6598 e 0.4359, respetivamente). Estes resultados sugerem que a arquitetura de ***deep learning*** foi mais eficaz na captura das relações complexas necessárias para estimar o valor contínuo da gorjeta, superando os modelos clássicos avaliados, incluindo o **MLP**, o **Random Forest** e o **SVM**.

6. Referências

[1] NYC Taxi & Limousine Commission. “TLC Trip Record Data”. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page